

Prácticas

2º cuatrimestre

Proyectos I
Ingeniería del Software
U-TAD

Guillermo casado
2019-2020

Práctica 11: Sensor de distancia Ultrasonidos

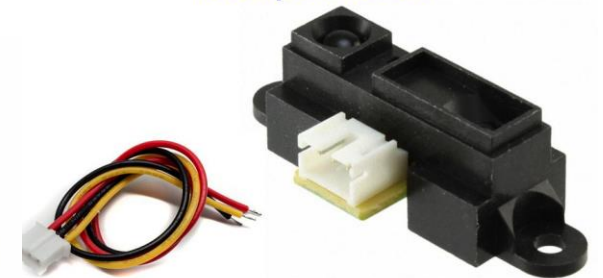


Sensores de distancia

- Hay multitud de sensores de distancia. En electrónica suelen utilizar estos dos tipos:
 - **Infrarrojos:** Sharp 2Y0A21. Utiliza luz infrarroja
 - **Ultrasonidos:** HC-SR04. Es el que vamos a utilizar



Sensor De Distancia Infrarrojo
10 - 80 Cm
Sharp 2Y0A21
Ideal para robots

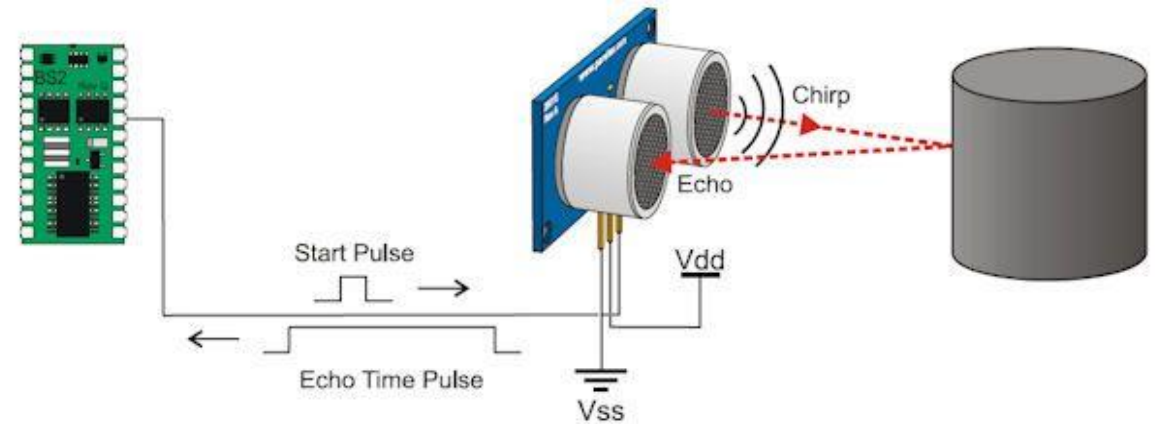
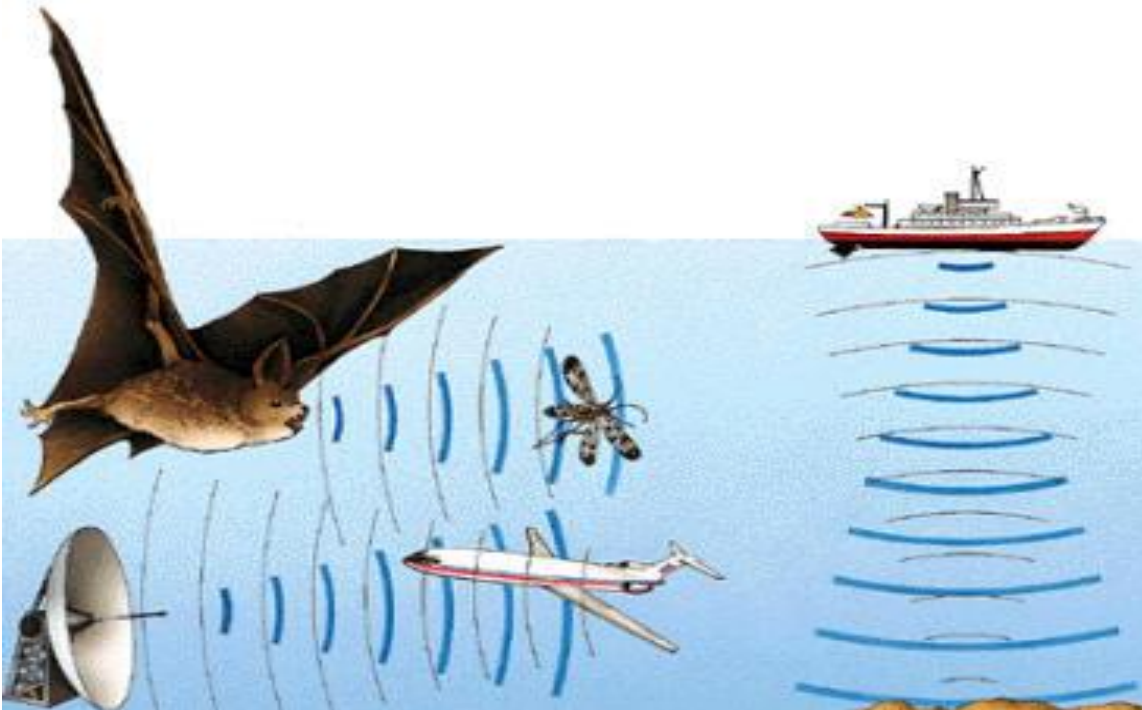


Características

Distancia de detección	10 a 80 cm
Tipo de salida	Analógica
Voltaje de alimentación	4.5 a 5.5VDC
Corriente de alimentación	40 mA
Voltaje de salida	400 mV @ 80cm

Sensor de Ultrasonidos

- El sensor emite una onda de presión en el rango de ultrasonido y queda a la espera de recibir su rebote. Mide el tiempo que ha tardado y calcula la distancia sabiendo la velocidad del sonido en el medio.



Sensor de Ultrasonidos

- Ejemplo.

La velocidad de sonido en el aire a temperaturas estándar (20º C, nivel del mar, y humedad relativa de 50%) es de 343 m/s

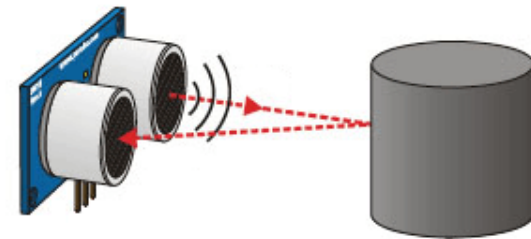
El sensor nos da los microsegundos entre la emisión y la recepción de un pulso.

Para calcular la distancia hay que convertir la velocidad a cm/microseg y dividir el tiempo para dos (ida y vuelta del pulso)

$$343 \frac{m}{s} \cdot 100 \frac{cm}{m} \cdot \frac{1}{1000000} \frac{s}{\mu s} = \frac{1}{29.2} \frac{cm}{\mu s}$$

$$Distancia(cm) = \frac{Tiempo(\mu s)}{29.2 \cdot 2}$$

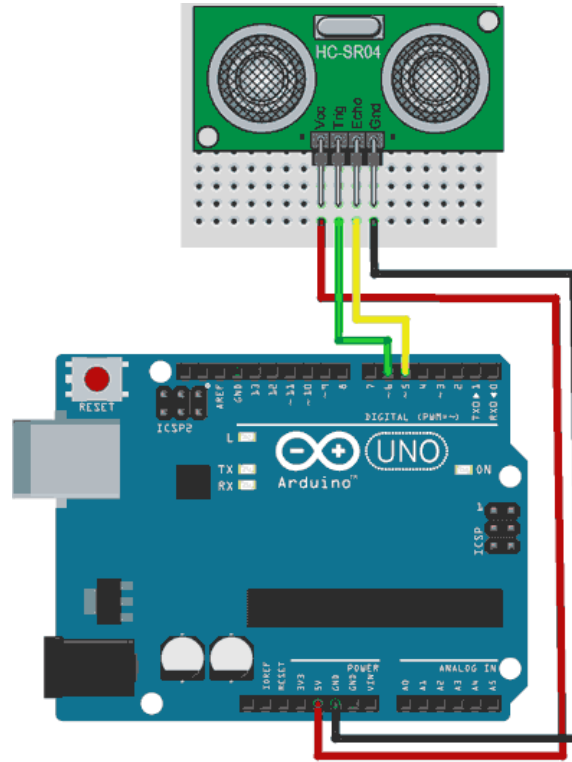
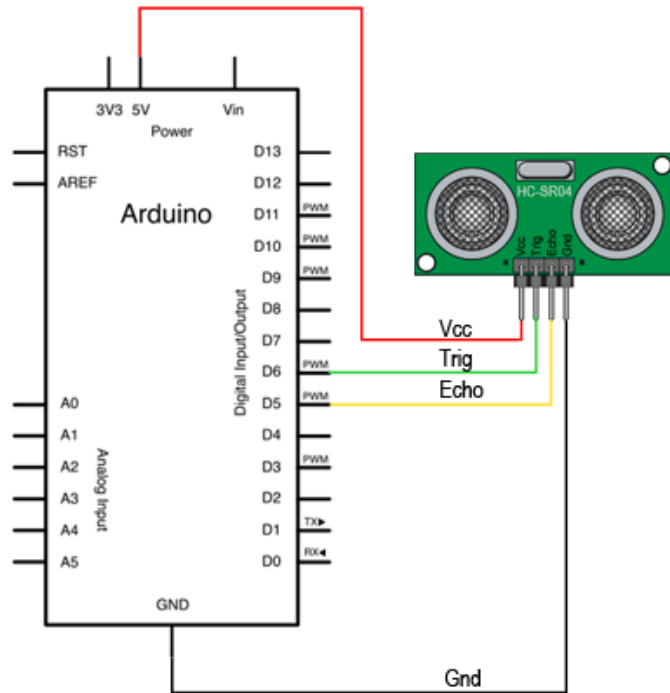
- Rango:
- Precisión:
- En muchos casos se utiliza como sensor de presencia.



$$Tiempo = 2 * (Distancia / Velocidad)$$

$$Distancia = Tiempo \cdot Velocidad / 2$$

Sensor de Ultrasonidos: Montaje Arduino



PIN 5V
PIN GND
PIN Trigger: envía señal
PIN Echo: recibe señal

Buen tutorial:

<https://www.luisllamas.es/medir-distancia-con-arduino-y-sensor-de-ultrasonidos-hc-sr04/>

Sensor de Ultrasonidos: Código

Modo Sin librería

Hay que medir el tiempo entre que se envía un pulso y se recibe la respuesta. Para ellos se pasa de LOW a HIGH en el PIN Trigger y se mide el tiempo que pasa hasta que el PIN Echo pasa a HIGH.

Con esta función basta llamar en loop() a la función ping() cada cierto tiempo:

```
int cm = ping(trigPIN, echoPIN); // trigPIN y echoPIN son los puertos digitales  
donde esta conectado el sensor
```

```
// ...
```

```
delay(1000);
```

```
int ping(int TriggerPin, int EchoPin) {  
    long duration, distanceCm;  
  
    digitalWrite(TriggerPin, LOW); //para generar un pulso limpio ponemos a LOW 4us  
    delayMicroseconds(4);  
    digitalWrite(TriggerPin, HIGH); //generamos Trigger (disparo) de 10us  
    delayMicroseconds(10);  
    digitalWrite(TriggerPin, LOW);  
  
    duration = pulseIn(EchoPin, HIGH); //medimos el tiempo entre pulsos, en microsegundos  
  
    distanceCm = duration * 10 / 292 / 2; //convertimos a distancia, en cm  
    return distanceCm;  
}
```

Sensor de Ultrasonidos: Código

- librería NewPing

```
#include <NewPing.h>

const int UltrasonicPin = 5;
const int MaxDistance = 200;

NewPing sonar(UltrasonicPin, UltrasonicPin, MaxDistance);

void setup() {
  Serial.begin(9600);
}

void loop() {
  delay(50); // esperar 50ms entre pings (29ms como minimo)
  Serial.print(sonar.ping_cm()); // obtener el valor en cm (0 = fuera de rango)
  Serial.println("cm");
}
```


Sensor de Ultrasonidos. Ejercicios

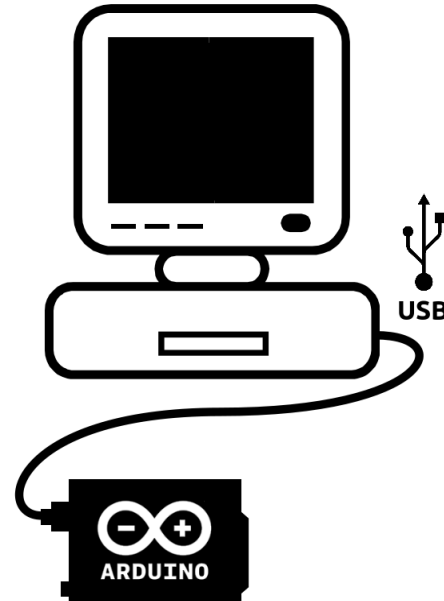
1. Devolver por el puerto serial el valor medido. Chequear la distancia mínima y la máxima que el sensor da una medida aceptable.
2. Programar el sistema de forma que encienda de 0 a 3 leds dependiendo de la distancia a la que encuentre un obstáculo.
3. Hacer que el brillo de los LEDs varíe gradualmente del 0% al 100% en el rango de distancias correspondiente

12

Práctica 12: Comunicación Serial

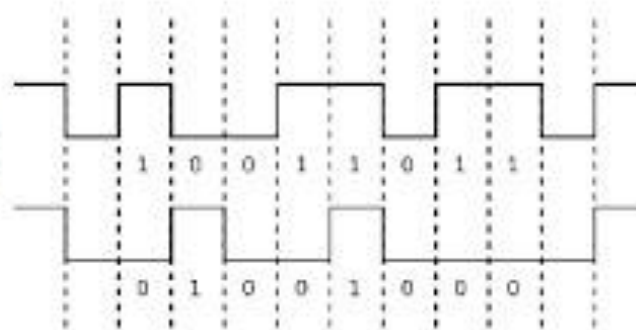


Computadora
anfitriona



Comunicación Serial

- El puerto USB de Arduino permite:
 - **Cargar programas** en Arduino desde una computadora
 - Dar **Energía eléctrica** al circuito
 - **Intercambiar (Enviar y Recibir) datos** entre el ordenador y Arduino. Esto permite controlar un circuito desde el ordenador, y/o una aplicación del ordenador desde un circuito de Arduino.



Comunicación Serial

Guión

- Envío de info desde Arduino: escribir en consola
- Recepción de info en Arduino (desde consola)
- Buffer

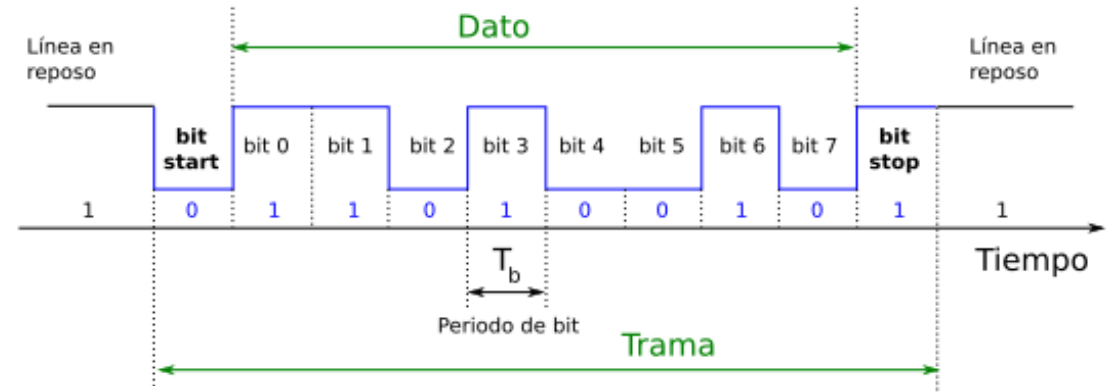
Comunicación

- **Velocidad de transmisión:**

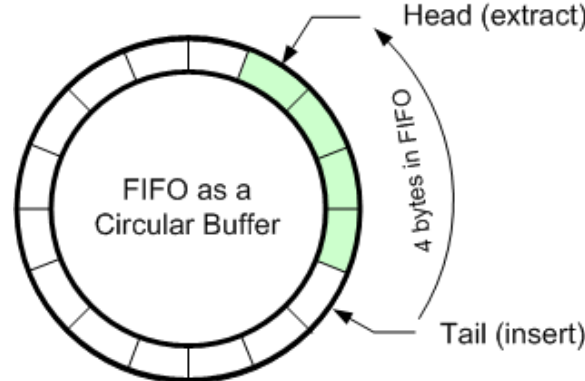
- 1 **baudio** = 1 bit/segundo
- Se utilizan velocidades estandar:
110, 300, 600, 1200, 2400,
4800, **9600**, **14400**, 19200,
38400, 57600, 115200,
128000 y 256000

- **Buffer:**

- Espacio de memoria en el que se almacenan datos que se acumulan con el tiempo. Del buffer se extraen los datos recibidos según se necesiten.
- Suelen tener un mecanismo de **FIFO: First In First Out**. Se leen primero los bits que primero llegaron. Una vez leídos desaparecen del buffer.
- Se utiliza continuamente en transmisiones en tiempo real: imagen, sonido, otros datos...



[Enlace: Referenciabaudios](#)



Comm: ARD => Ordenador

- Referencia [Serial](#)
- Escribir **caracteres de texto ASCII**: [print\(\)](#), [println\(\)](#)
Es legible por los humanos
- Escribir **bytes**: [write\(\)](#)
Escribe en binario
- Interesa hacer [delay\(\)](#) después de enviar información. Debería ser más largo cuanto más información se envíe. `delay(5)`, por ejemplo vale para 5 bytes (40 bits) a 9600 bauds.
- Lo podemos leer por **consola** o por cualquier otro **programa** que haya establecido previamente una **conexión Serial**. Esto nos permite controlar un programa desde un interface basado en Arduino.

Comm: Ordenador => ARD

- Es posible comunicarse desde una aplicación. Por ahora nos comunicaremos desde la **consola** de puerto serie de Arduino.
- Podemos enviar **comandos** para pedir información al Arduino, controlar su circuito asociado...
- En consola podemos escribir **caracteres ASCII**, así que en Arduino recibiremos bytes correspondientes a caracteres ASCII.

Comm: recepción de datos en ARD

- Detectar si hay datos en el **buffer**:
`Serial.available()` ==> True/False
- Leer datos y utilizarlos.
 - `.parseInt()`: lee hasta que encuentra un entero y lo devuelve. `.parseFloat()`
 - `.read()`: lee el dato siguiente en el buffer.
 - `.readString()`: lee el dato siguiente en el buffer y lo guarda en un String.
- Leer datos hasta encontrar un caracter especial: **delimitador**.
 - `.readStringUntil(char)`: lee hasta un caracter dado y lo guarda en un String.

Ejemplos de Arduino

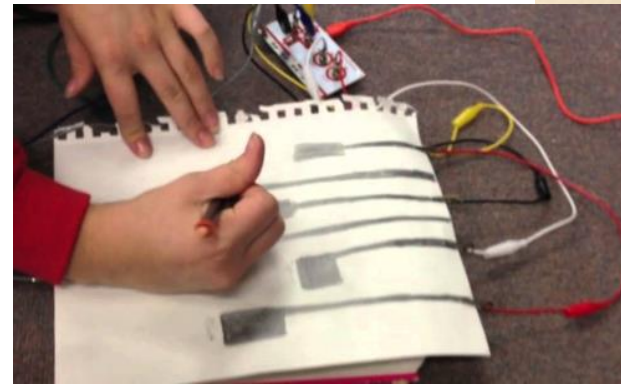
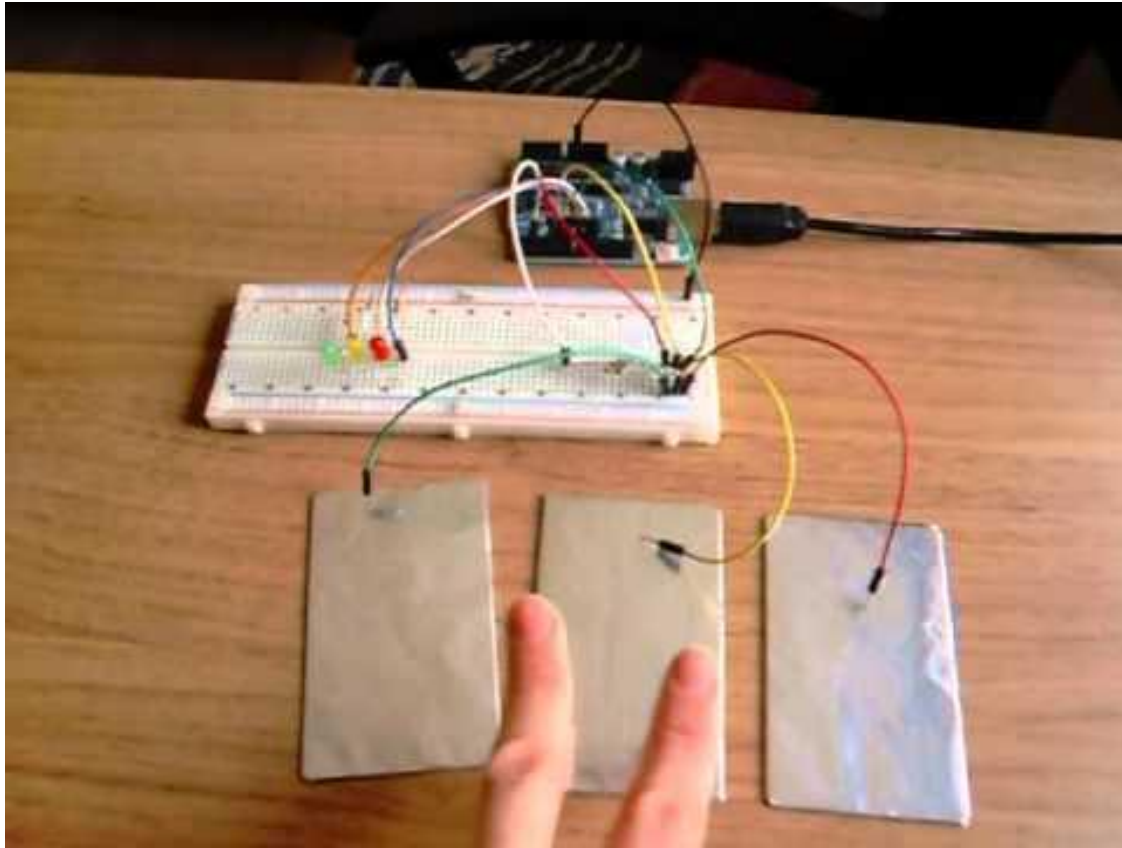
- **ASCII Table:** demo de cómo escribe Arduino en el serial y cómo trata los caracteres ASCII. `print()`, `println()`.
- **ReadASCIIString:** Lee texto y lo pasa a entero. Se utiliza con Processing.
- **SerialCallResponse** y **SerialCallResponseASCII:** Muestran un mini protocolo para establecer la comunicación.
- **Midi:** Arduino envía datos de sonido MIDI
- **Dimmer:** Arduino recibe bytes

Ejercicios

- Preparar un circuito para controlar 3 LEDs con PWM.
- Hacer que con las teclas se controlen los LEDs:
 - 1,2,3, encienden o apagan los LEDs correspondientes.
 - 4: parpadean los LEDs
 - 5: informa del estado de los LEDs
- Conectar el sensor de ultrasonidos y configurar para que tenga dos modos de reacción, como en práctica 10. Con las teclas A y B determinar el modo de trabajo

13

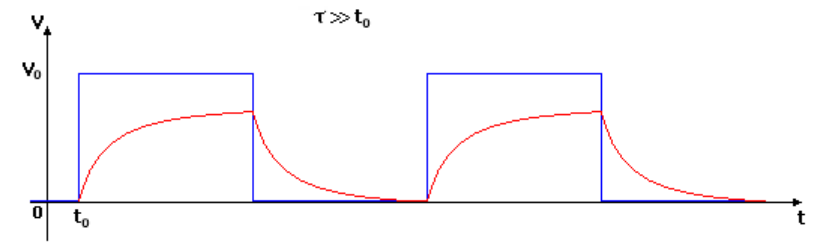
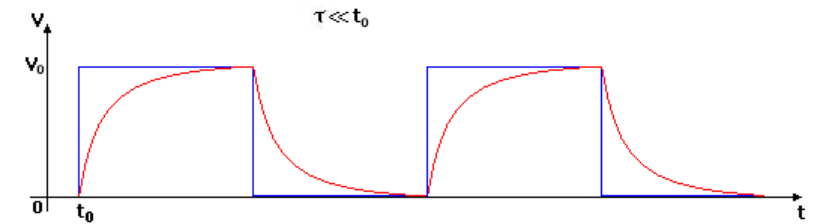
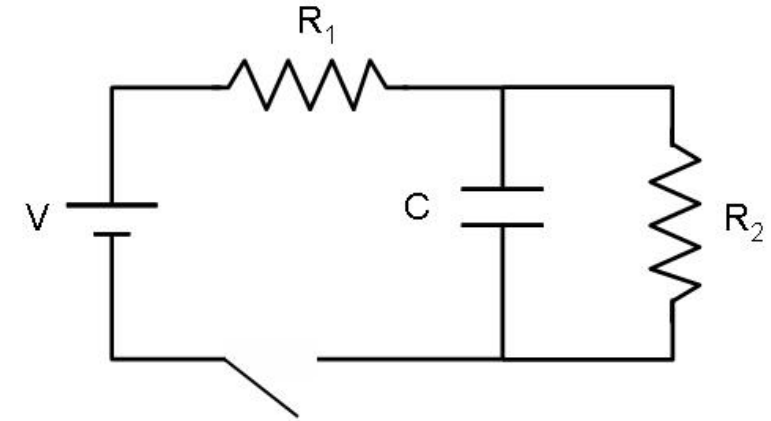
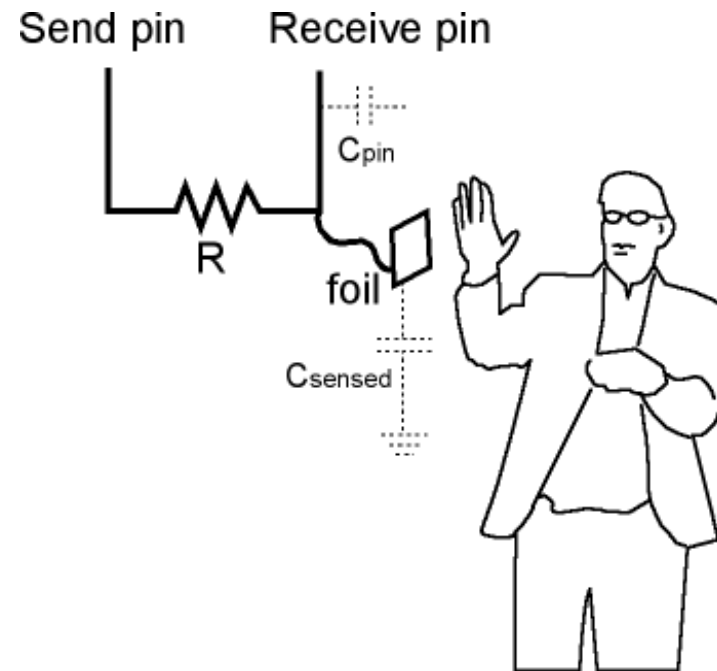
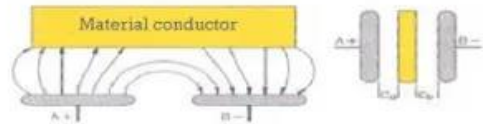
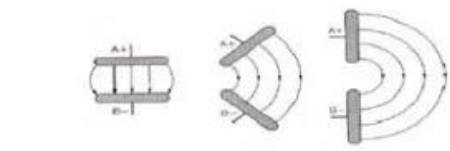
Práctica 13: Sensor Capacitivo DIY



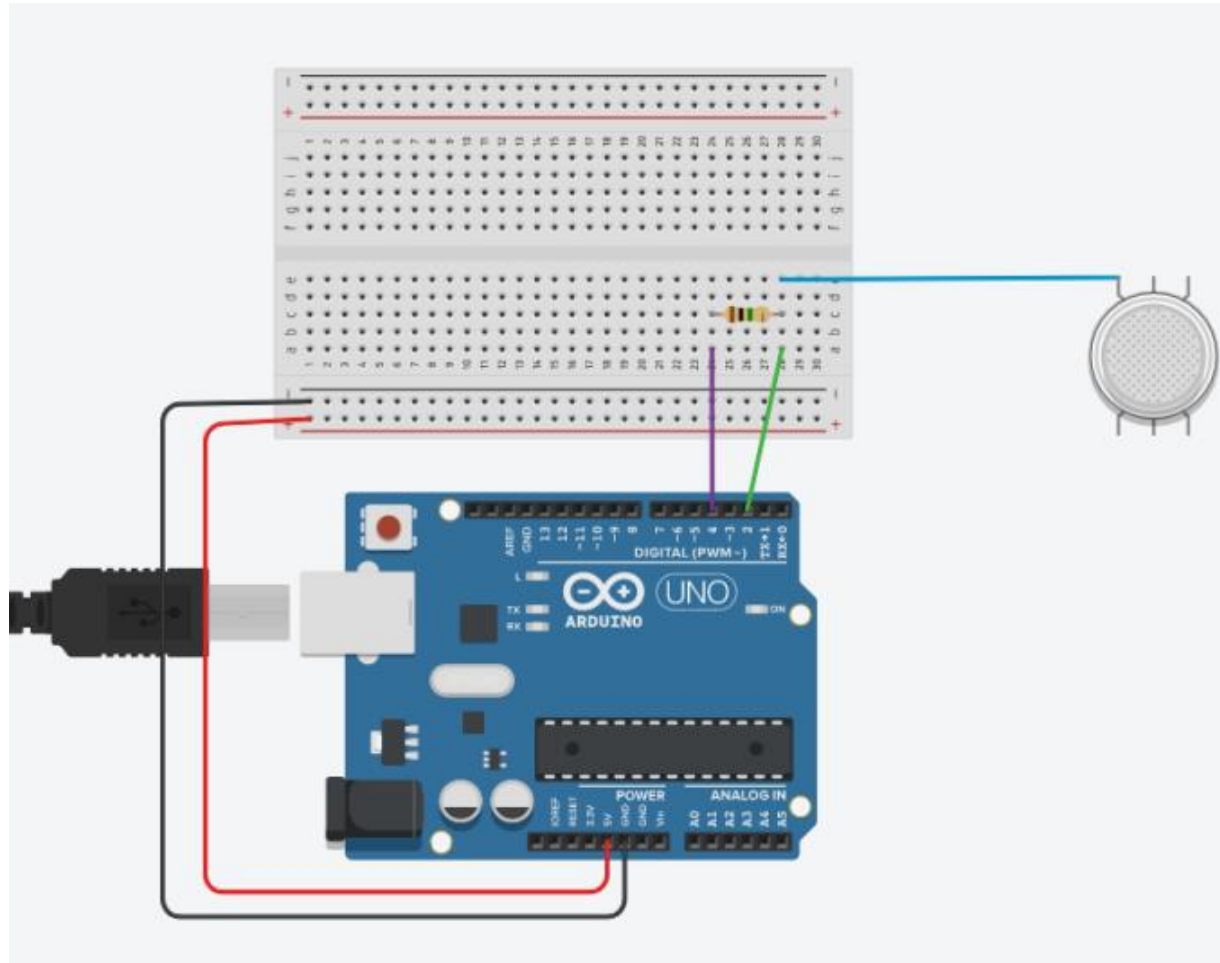
Efecto Capacitivo

- Permite detectar la cercanía y el contacto de un objeto (persona, planta...)
- Se basa en que entre dos medios cargados se puede acumular carga (condensador) y eso afecta a los circuitos que puedan estar conectados al sistema.
- Existen sensores comerciales de diferentes características, pero podemos **construir** uno nosotros mismos. DIY!
- Se puede convertir en un **sensor de presión** cubriendo de papel u otro material aislante la placa de aluminio. No es fácil.

Efecto Capacitivo

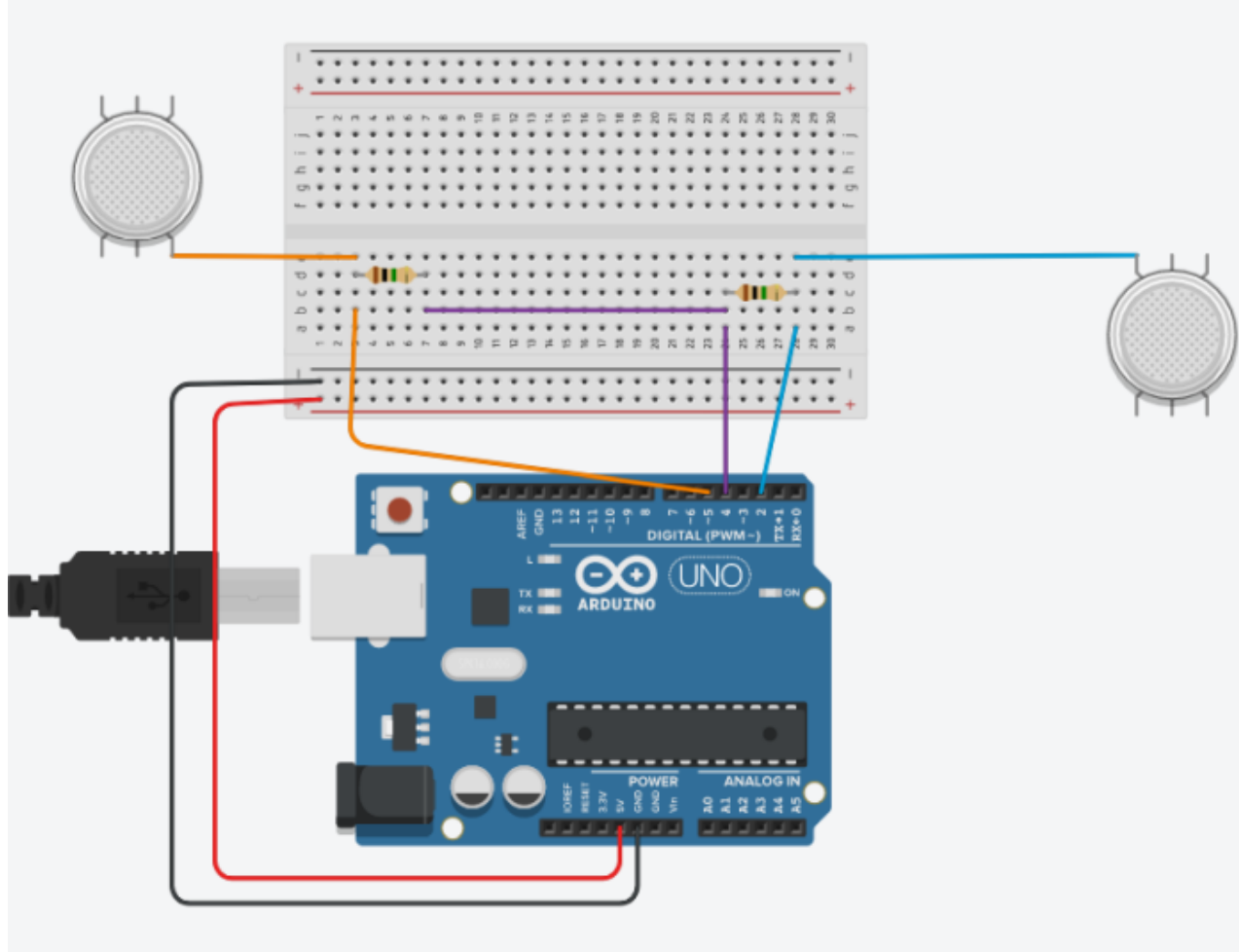


Efecto Capacitivo: Circuito



La resistencia es de
1MegaOhm o más

Efecto Capacitivo: Circuito



La resistencia es de
1MegaOhm o más

Si se ponen varios
sensores se comparte un
pin (el de transmisión)
entre todos.

Librerías

- Capacitive Sensor (<https://playground.arduino.cc/Main/CapacitiveSensor/>)
Usa 2 pines digitales
(<https://www.rinconingenieril.es/sensor-capacitivo/>)
- Otras:
 - ADCTouch y ADCTouch Sensor. Usan 1 pin Alog
 - Analog Touch (<https://github.com/NicoHood/AnalogTouch>)

Código

```
#include <CapacitiveSensor.h>
```

```
CapacitiveSensor  capSensor_4_2 = CapacitiveSensor(4,2); // 4,2 pin Tx, Rx
```

```
void loop() {
```

```
    // leer sensor capacitivo
```

```
    int valCap = capSensor_4_2.capacitiveSensor(32); // 30 es el numero de muestras que lee para  
                                                    // dar un valor
```

```
    Serial.println(valCap);
```

```
}
```

A partir de aquí ya podemos utilizar el resultado para imponer un umbral y decir si se ha sobrepasado o no.

Código

```
#include <CapacitiveSensor.h>
```

```
CapacitiveSensor  capSensor_4_2 = CapacitiveSensor(4,2); // 4,2 pin Tx, Rx  
int umbral = 100;
```

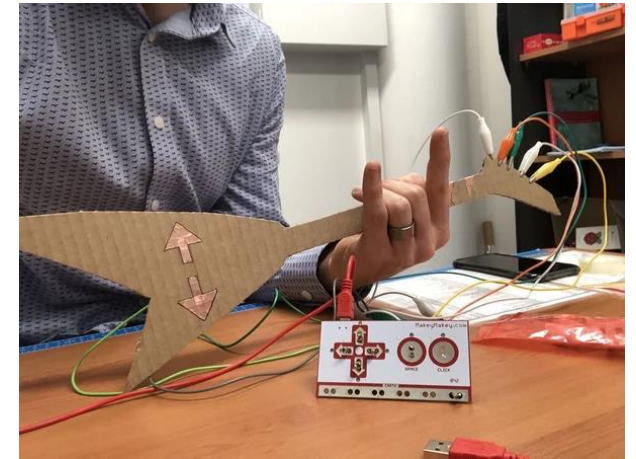
```
void loop() {  
    // leer sensor capacitivo  
    int valCap = capSensor_4_2.capacitiveSensor(32); // 30 es el numero de muestras que lee para  
                                                    // dar un valor  
  
    if(valCap>umbral) {  
        ...  
    }  
}
```

Actividad

- **Montar el circuito** del sensor capacitivo
- **Programar** la detección básica
- Desde el **Plot Monitor**, comprobar que funciona. **Experimentar** con diferentes **tamaños** del sensor, **materiales u objetos**, **distancia** de la mano...
- **Conectar** un **actuador** (LED, zumbador...) y **chequear** que se puede controlar con el sensor (ON/OFF)
- **Xtra:** Controlar el umbral con mensajes desde la consola Serial, o montar varios sensores y diferentes outputs...

Ejemplos y referencias

- Makey-Makey (<https://makeymakey.com/>)



- <https://www.luisllamas.es/interruptor-touchless-con-arduino-y-sensor-capacitivo/> (con sensor comercial)
- <https://www.ingmecafenix.com/automatizacion/sensor-proximidad-capacitivo/>

14

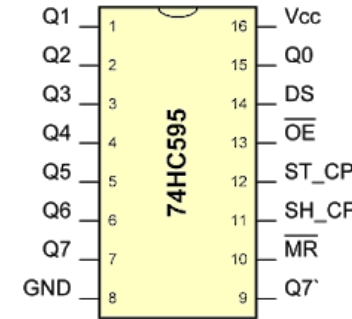
Cambio de registro 74HC595

Aumentar número de outputs digitales

Cap 16 Libro de Proyectos de Elegoo

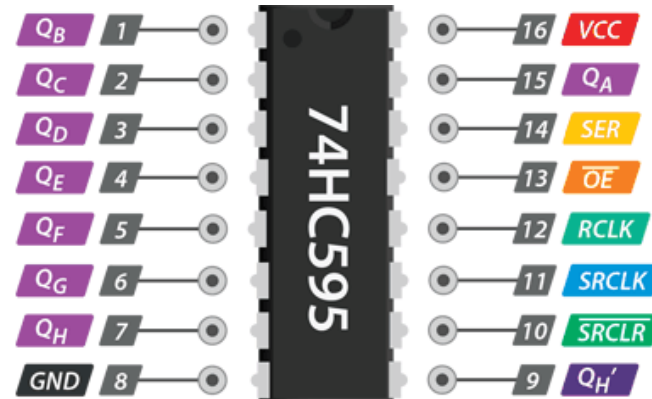
74HC595 Shift register de 8 bits

- Dispositivo que permite multiplicar una salida digital hasta por 8 salidas digitales.
- Tiene un puerto serie de **entrada** (DS) y 8 puertos de salida en **paralelo** (Q1, Q2, Q3...).
- Se controla con 3 Pines digitales:
11-SH_CP (SRCLK),
12-ST_CP (RCLK),
14-DS (SER)
y: Vcc (5V) y GND

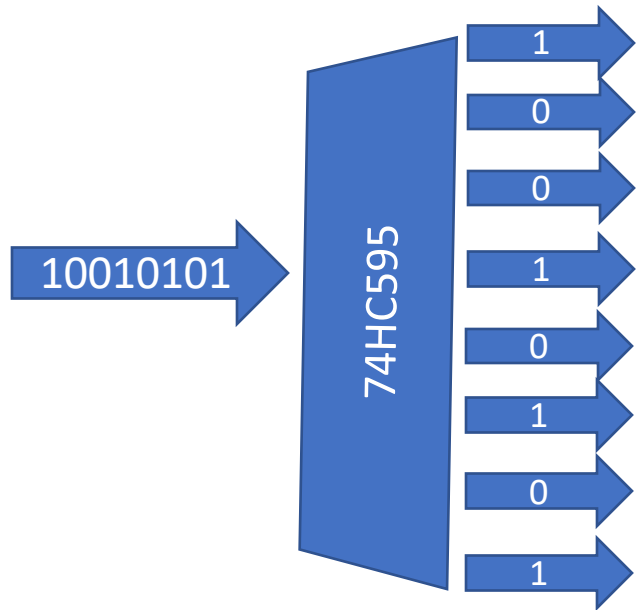


74HC595 8-Bit Shift Register Pinouts

Pin	Symbol	Description
1	Q1	Parallel data output (bit-1)
2	Q2	Parallel data output (bit-2)
3	Q3	Parallel data output (bit-3)
4	Q4	Parallel data output (bit-4)
5	Q5	Parallel data output (bit-5)
6	Q6	Parallel data output (bit-6)
7	Q7	Parallel data output (bit-7)
8	GND	Ground (0 V)
9	Q7'	Serial Data Output
10	MR	Master Reset (Active Low)
11	SH_CP	Shift Register Clock Input
12	ST_CP	Storage Register Clock Input
13	OE	Output Enable (Active Low)
14	DS	Serial Data Input
15	Q0	Parallel data output (bit-8)
16	Vcc	Positive Supply Voltage

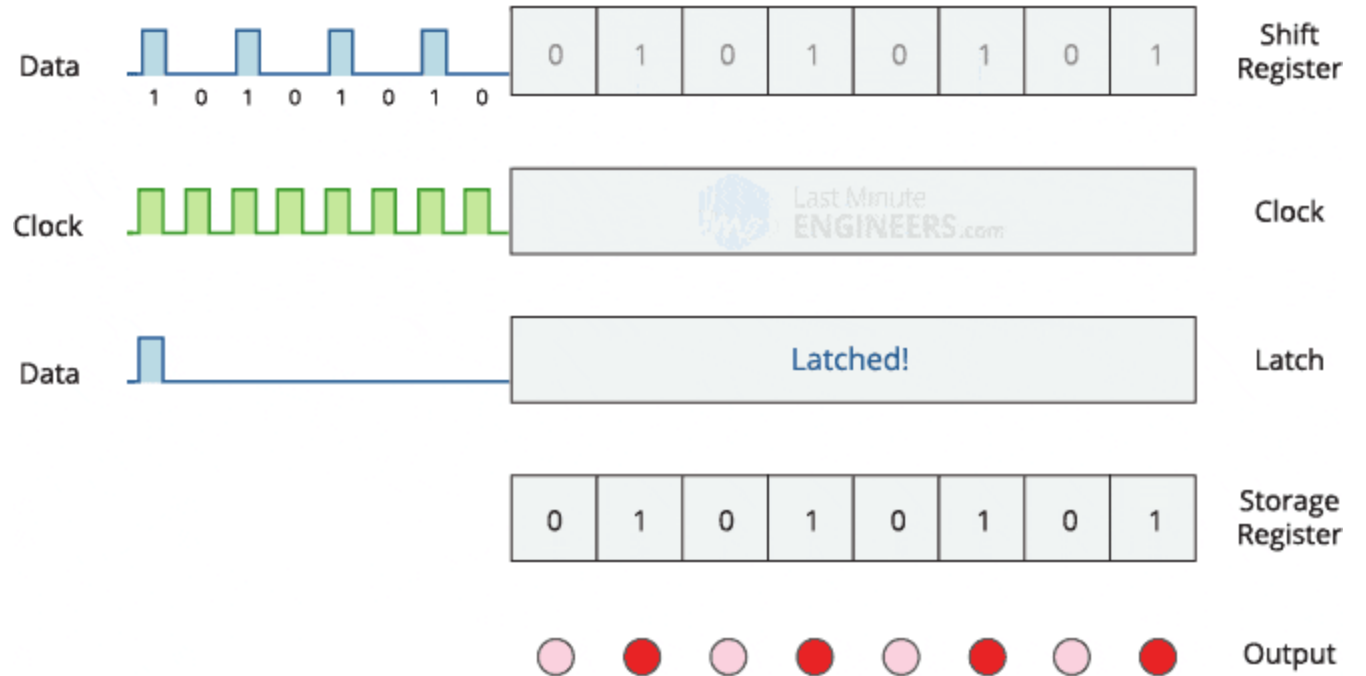


Shift Register



INPUT
(Serie)

OUTPUT
(Paralelo)

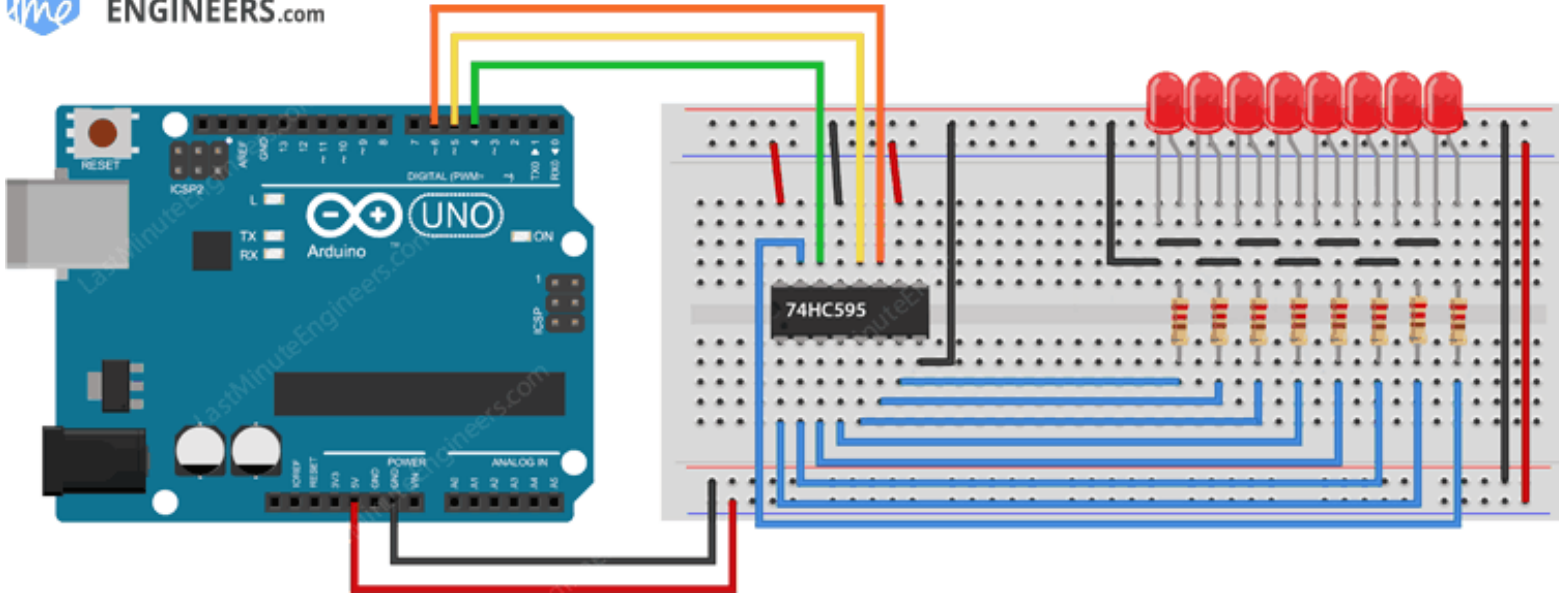


[enlace a AnimaciónShiftRegister](#)

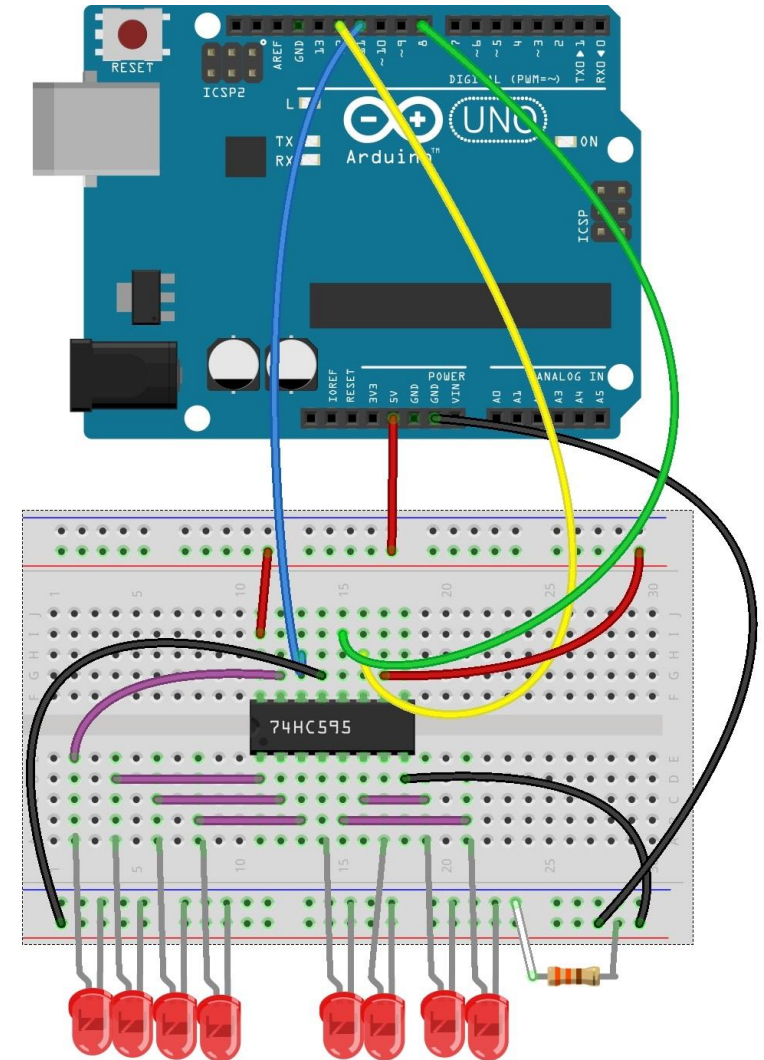
Shift Register: Actividad

- Montar el circuito del shift register para controlar 8 LEDs
- No hace falta cargar librerías. Arduino IDE ya tiene las **funciones necesarias para trabajar con bytes**:
 - `setBit()`
 - `shiftOut()`
- Programar el ejemplo incluido en esta presentación.
- **Programar el encendido y apagado de los LEDs mediante comandos serial:** Por ejemplo:
 - con "I" se indica que se encienda un bit dado: *Inbit*, con *nbit* el número de bit: 0-7
 - Con "Z" se indica que se apague un bit dado: *Znbit*, con *nbit* el número de bit: 0-7
- Se puede utilizar para visualizar números en binario.

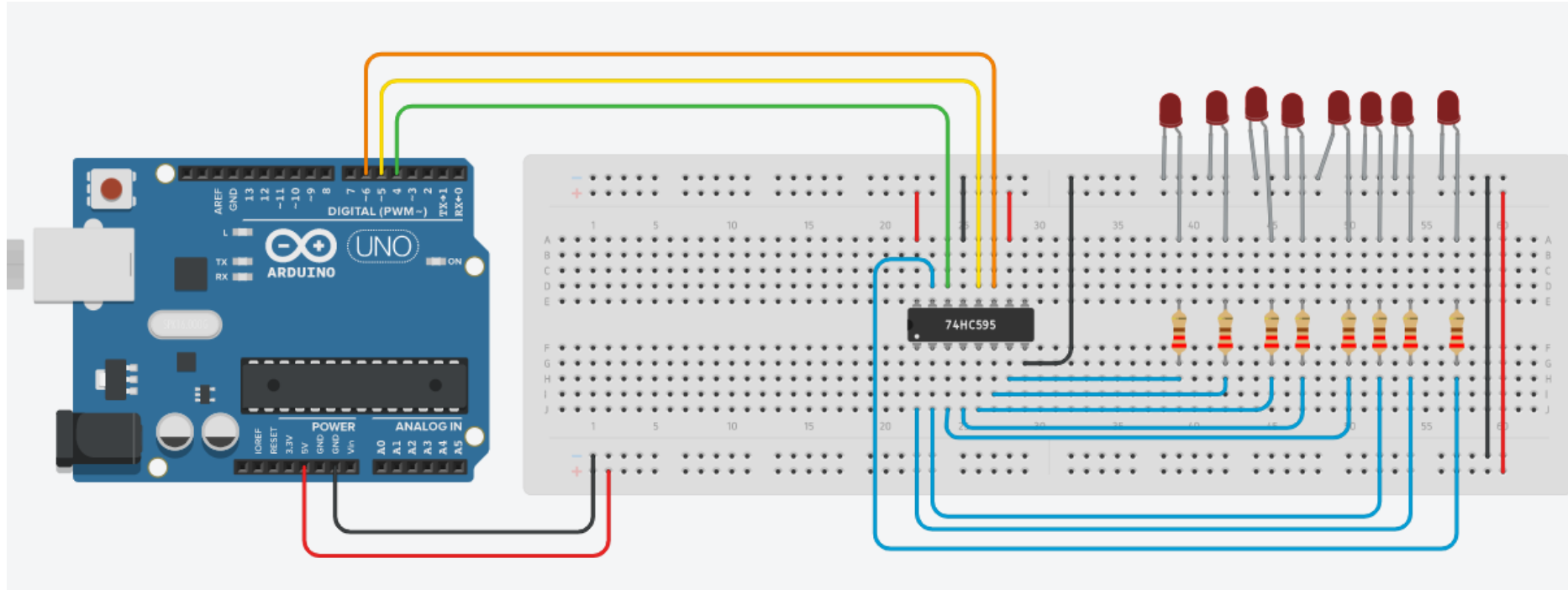
Shift Register: Circuito



MEJOR ESTE!



Shift Register: Circuito



MÁS SIMPLE

Shift Register: Código

- `bitSet()`:

`bitSet(x,n)`

Pone en 1 el bit número n del valor numérico x.

Lo contrario (ponerlo a 0) se hace con `bitClear(x,n)`

- `shiftOut()`:

`shiftOut(dataPin, clockPin, bitOrder, value)`

envía por un pin (`dataPin`) un byte (`value`) bit a bit. De izda a dcha. Escribe el bit en el pin de data y marca una señal (HIGH to LOW) en el pin de reloj (`clockPin`) para indicar que el bit esta disponible.

`bitOrder` = MSBFIRST o LSBFIRST (Most/Least Significant Byte First)

Shift Register: Código

- **Variables globales:**

- 3 para guardar los pines digitales utilizados: `pinData`, `pinClock`, `pinLatch`
- 1 variable tipo byte para guardar los 8 bits a transferir
 - `byte leds = 0;`

- **setup() :**

- Inicializar los 3 pines digitales para output
- Inicializar Serial

Shift Register: Código

- **Función `updateShiftRegister()`:**

Esta función realiza el paso de los bits que componen la variable tipo byte `leds` a la vez que señala en el `pinClock` y `pinLatch` la transferencia de información.

Se puede hacer que tenga un parámetro para el valor a convertir y que no fuese siempre la variable `leds`:

```
updateShiftRegister(byte _leds)
```

Incluso que recibiera como parámetros los pines digitales utilizados:

```
updateShiftRegister(byte _leds, int _nD, int _nC, int _nL)
```

```
void updateShiftRegister()  
{  
    digitalWrite(latchPin, LOW);           // inicio transmision  
    shiftOut(dataPin, clockPin, LSBFIRST, leds); // pasa los bits al data  
                                           // y marca en pinClock  
    digitalWrite(latchPin, HIGH);          // fin transmision  
}
```

Shift Register: Código

En este ejemplo se iluminan 1 a 1 los leds

- `loop()` :

```
// Initially turns all the LEDs off,  
// by giving the variable 'leds' the value 0  
leds = 0;  
  
updateShiftRegister();  
delay(500);
```

```
// Turn all the LEDs ON one by one.
```

```
for (int i = 0; i < 8; i++)  
{  
    // Set the bit that controls that LED in  
    // the variable 'leds'  
    bitSet(leds, i);  
    updateShiftRegister();  
    delay(500);  
}
```

```
Serial.println("FIN LOOP");
```

Shift Register. Referencias y más

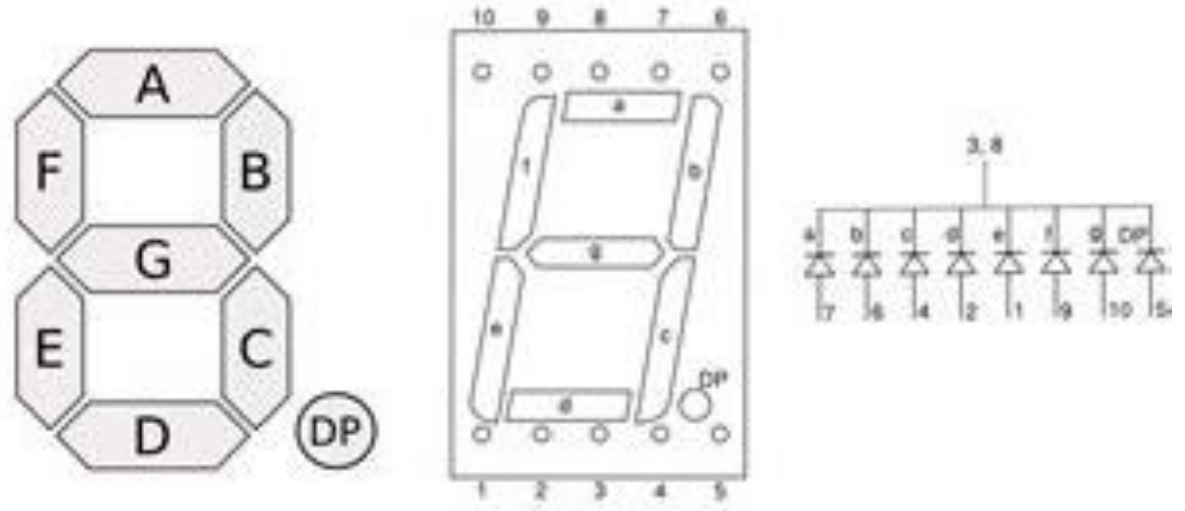
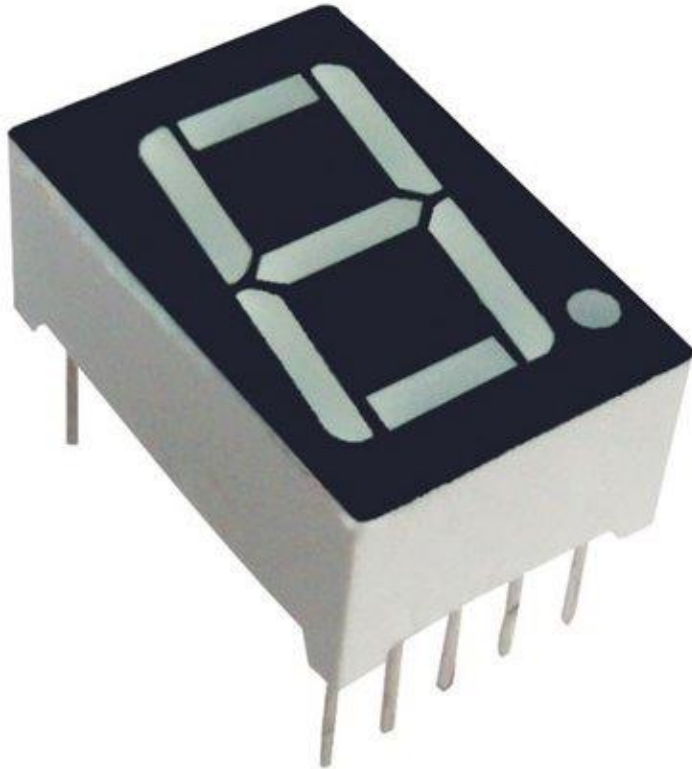
- El **74HC595** es un shift register de 1 byte. También existen de 16, 32, 64 bits...
- También existen dispositivos que actúan al revés: unifican múltiples inputs paralelos en uno input serial (**74HC165**)
- <https://www.prometec.net/shift-registers/>
- <https://lastminuteengineers.com/74hc595-shift-register-arduino-tutorial/>
- <https://www.electrontools.com/Home/WP/2016/03/09/registro-de-desplazamiento-74hc595/> (shiftReg y 7-segment)

15

7 Segment Display

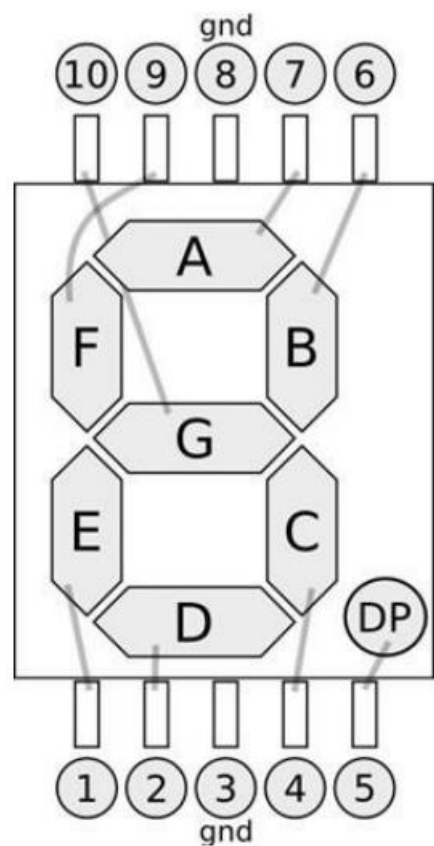
Cap 19 Libro de Proyectos de Elegoo

7 Segment Display



Cada segmento está relacionado con una patilla.
Además, hay dos patillas para la tierra.

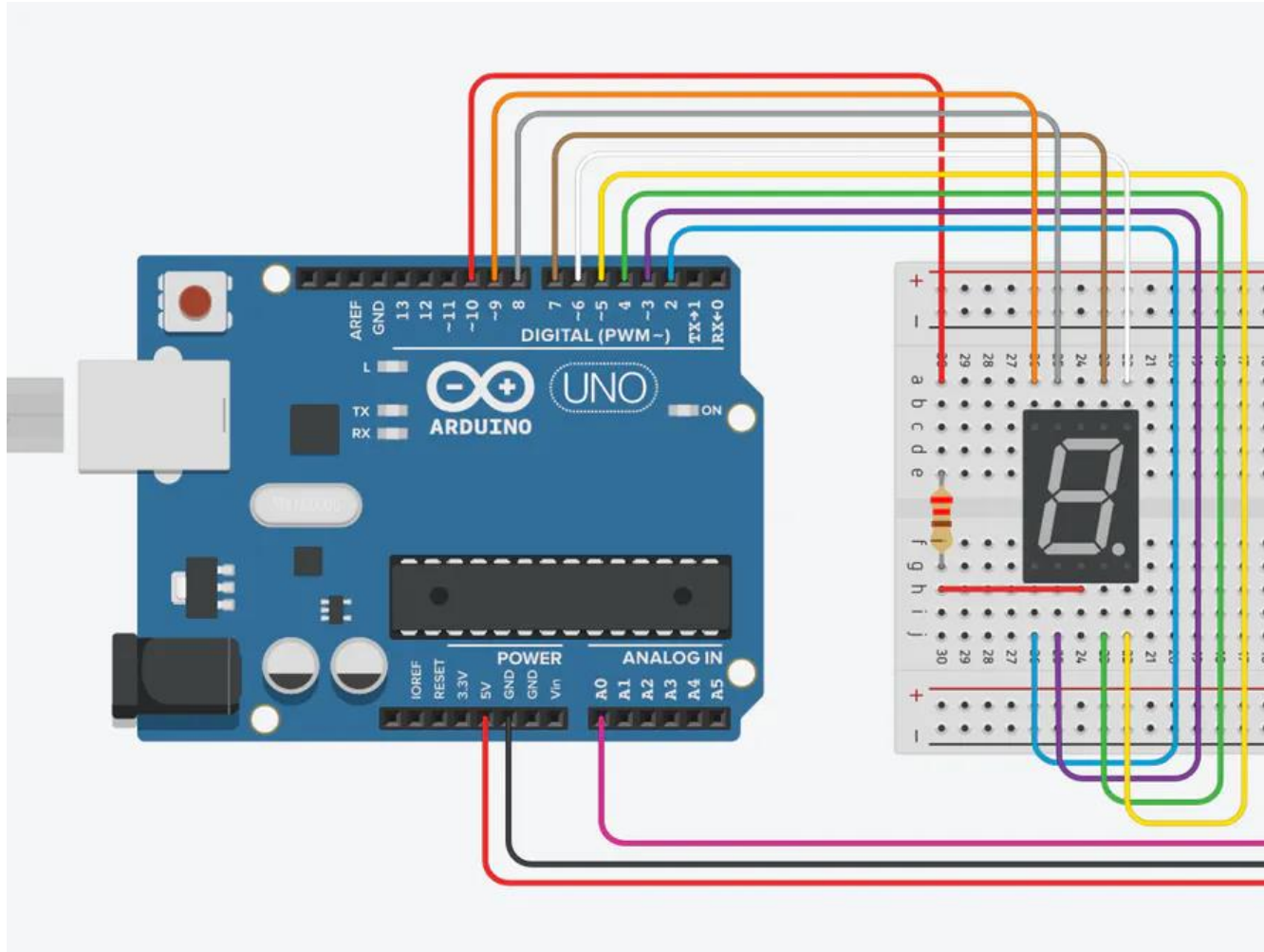
Existen dos tipos: con **ánodo común** o con **cátodo común**. El que utilizamos es de **Cátodo común**.



En la tabla se muestra
qué segmentos deben
estar activos para
representar los diferentes
números del 0 al 9

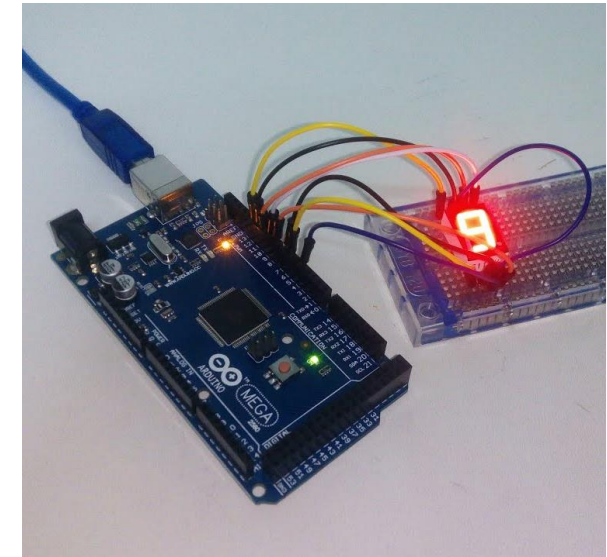
Display digital	dp	a	b	c	d	e	f	g
0	0	1	1	1	1	1	1	0
1	0	0	1	1	0	0	0	0
2	0	1	1	0	1	1	0	1
3	0	1	1	1	1	0	0	1
4	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1
6	0	1	0	1	1	1	1	1
7	0	1	1	1	0	0	0	0
8	0	1	1	1	1	1	1	1
9	0	1	1	1	1	0	1	1

Conexión directa Arduino <-> 7-segment (alone)

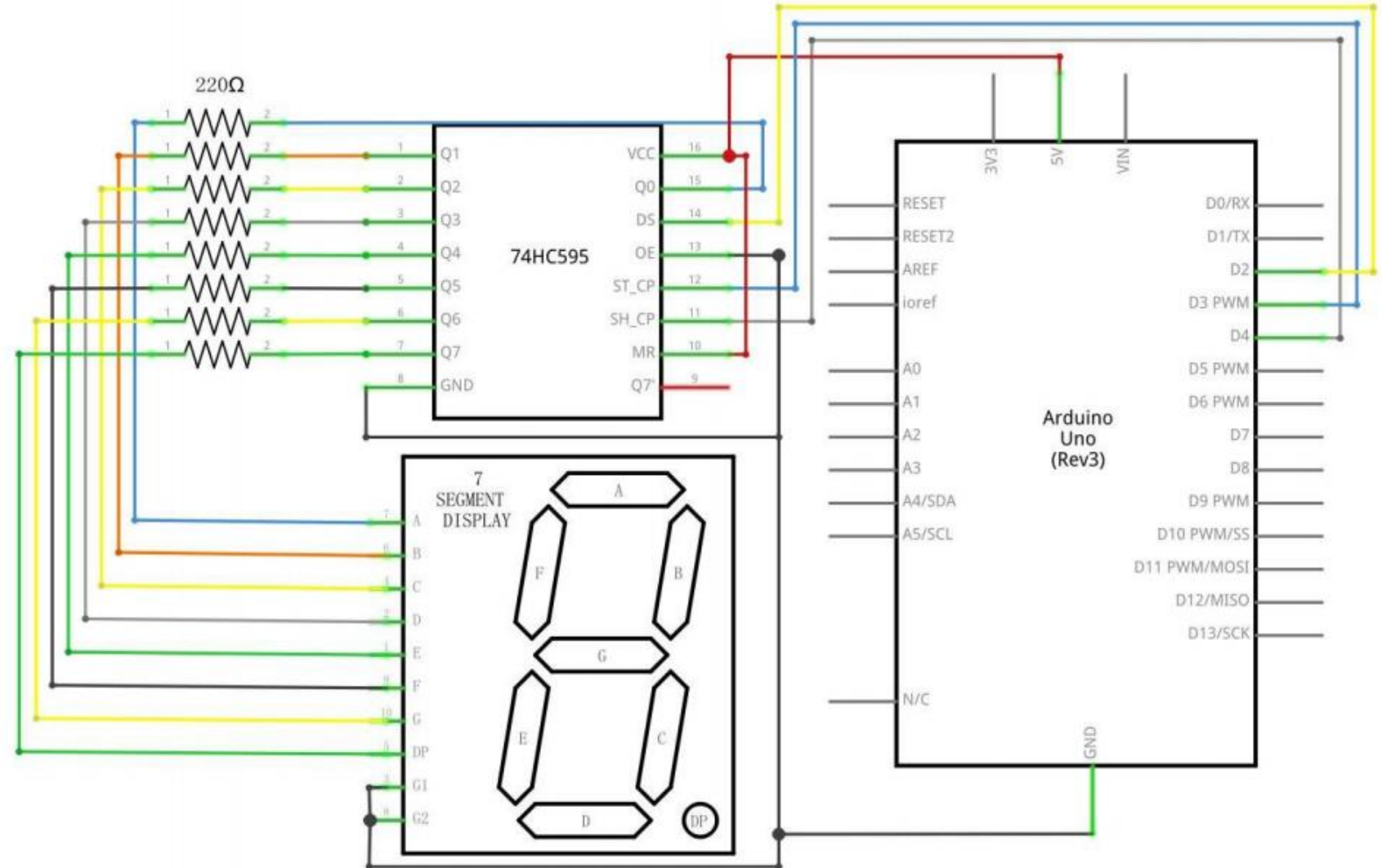
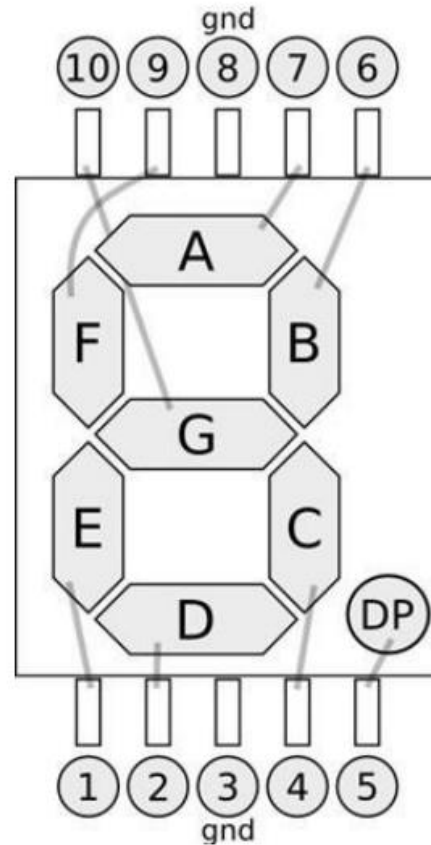


Se necesitan **8 pines digitales** para controlar el módulo 7-segment.

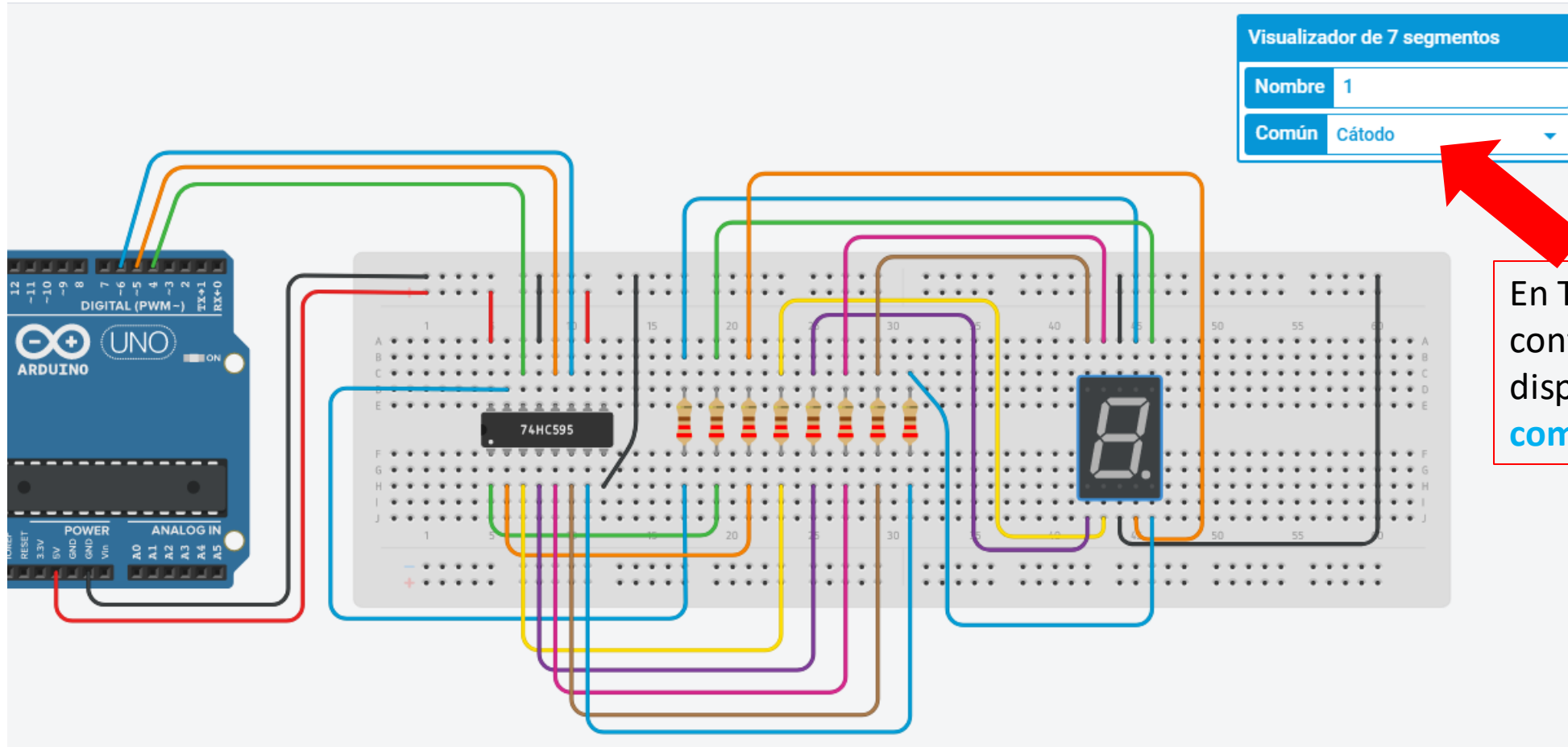
Vamos a utilizar un módulo 74HC595 para utilizar solo **3 pines digitales**.



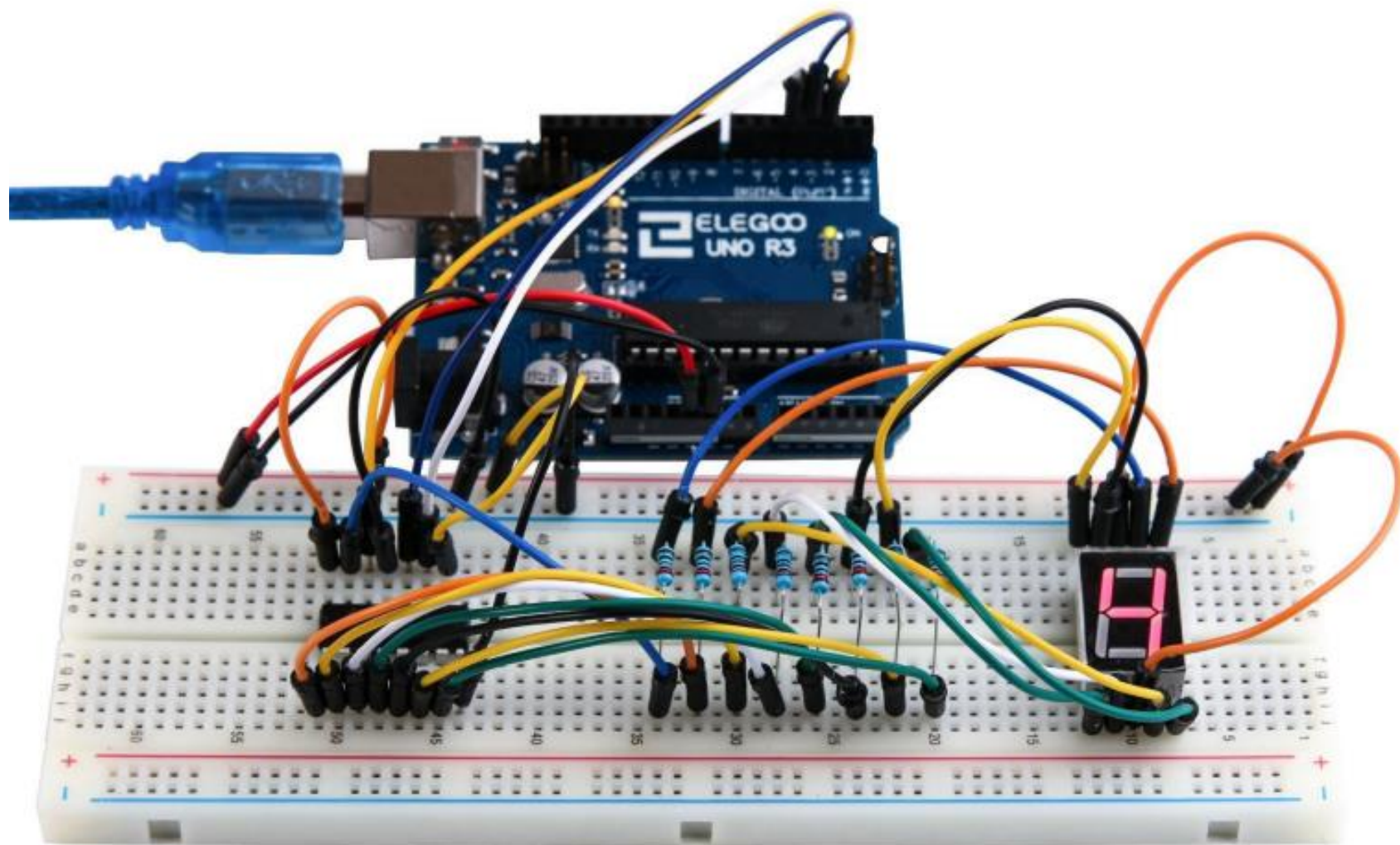
7 Segment: Esquema de conexiones con 74HC595



Circuito 7-segment con Shift Register



En Tinkercad hay que configurar el 7-segment display como **Cátodo común**.



Actividad

- Montar el circuito del 7-segment con el Shift Register.
- Subir el código adjunto. Debería mostrar los números del 9 al 0, uno cada 5 segundos.
- Hacer que muestre en pantalla una cifra que se halla pasado por Serial. Si se ponen varias, coger la última.

Código

- Dos partes importantes:
 - I. Definición de números
 - II. Función que escribe en el display

I. Definición de números:

```
byte seven_seg_digits[10] = { B11111100,  // = 0
                               B01100000,  // = 1
                               B11011010,  // = 2
                               B11110010,  // = 3
                               B01100110,  // = 4
                               B10110110,  // = 5
                               B10111110,  // = 6
                               B11100000,  // = 7
                               B11111110,  // = 8
                               B11100110   // = 9
                               };
```

Código

II. Función que escribe en el display:

```
void sevenSegWrite(byte digit) {  
    // ponemos el pin Latch en LOW antes de enviar datos  
    digitalWrite(latchPin, LOW);  
    // Enviamos los datos asociados al digit (bit pattern)  
    shiftOut(dataPin, clockPin, LSBFIRST, seven_seg_digits[digit]);  
    // ponemos el pin Latch en HIGH despues de enviar datos  
    digitalWrite(latchPin, HIGH);  
}
```

Referencias

- Práctica 19 del libro de proyectos de Elegoo. Se puede encontrar entre los ejemplos de Arduino si se descarga el pack de código de **Starter Kit Elegoo**.
- <https://www.electrontools.com/Home/WP/2016/03/09/display-7-segmentos/>

16

Emisor-Receptor IR

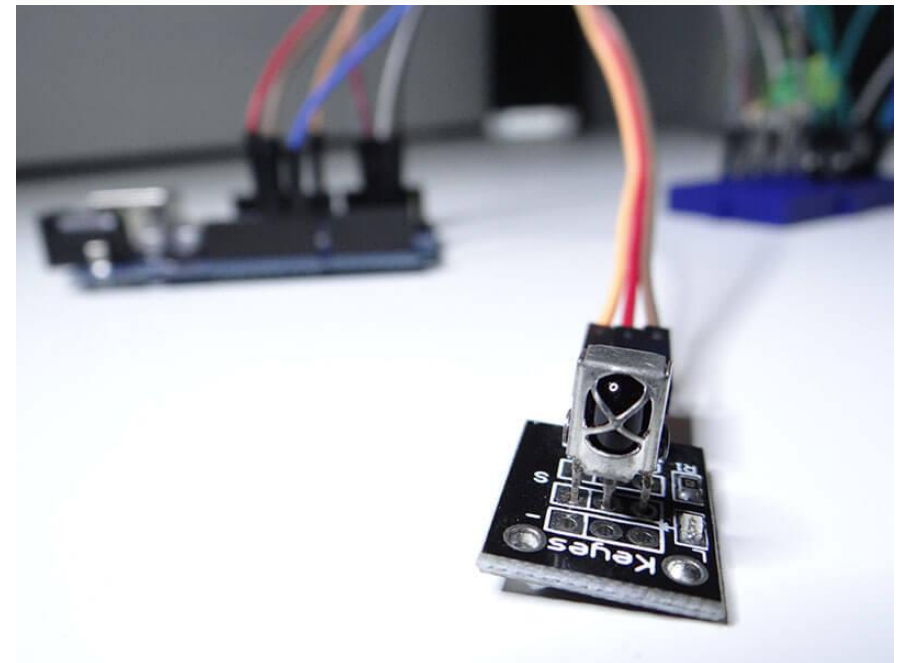
Control remoto por IR

Cap 13 Libro de Proyectos de Elegoo

Guión

- Luz IR: frecuencia, banda
- Transmisión wireless
- Transformación sensor => Arduino
- Código con Librería "**IRremote.h**"
- Obtener códigos para cada tecla:
`Serial.println(results.value, HEX);`
- Actividad: 4 LEDs o se puede utilizar también el montaje de 7-Segment
`digitalWrite(pin, !digitalRead(pin));`
- Referencias

Emisor-Receptor IR

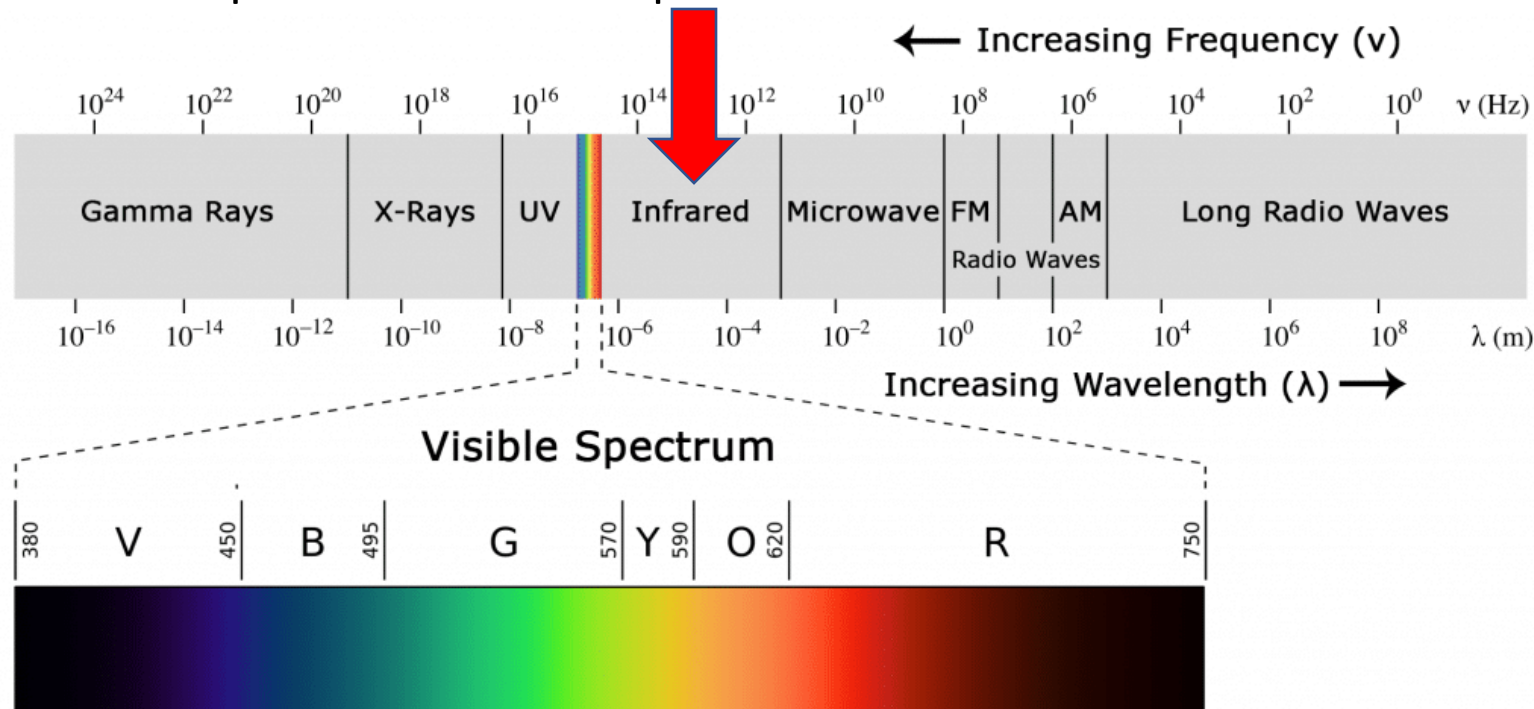


Infrarojo

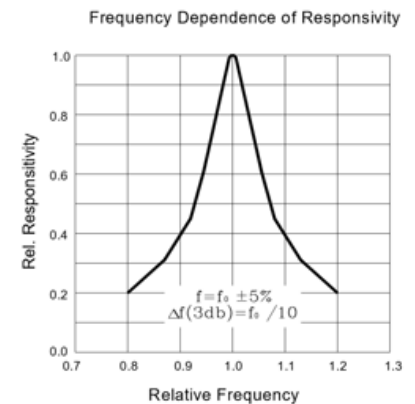
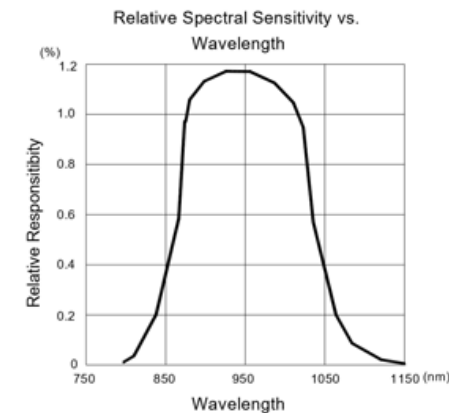
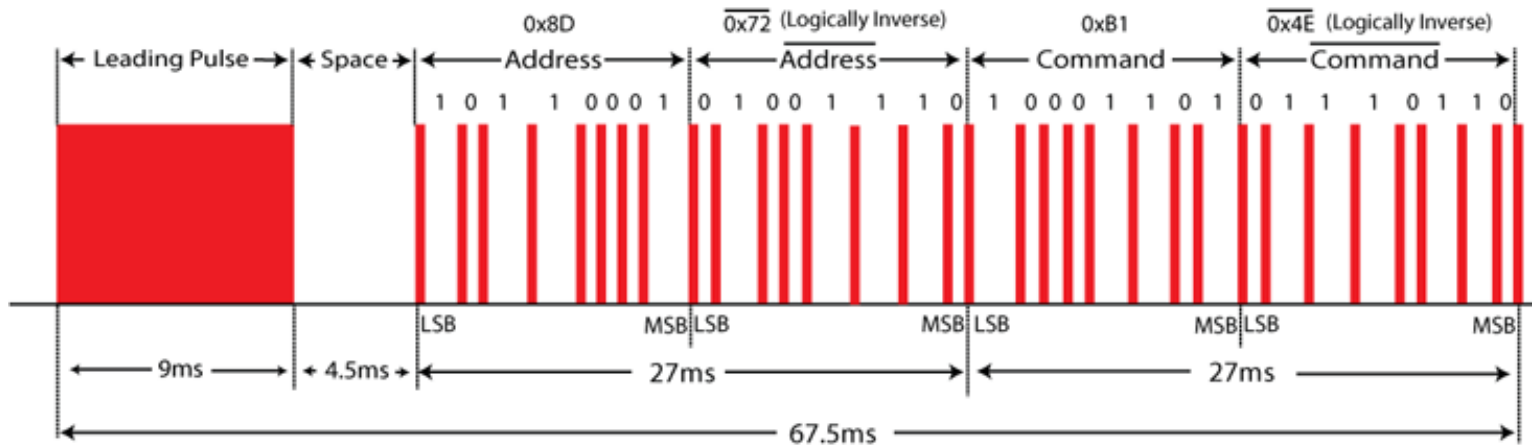
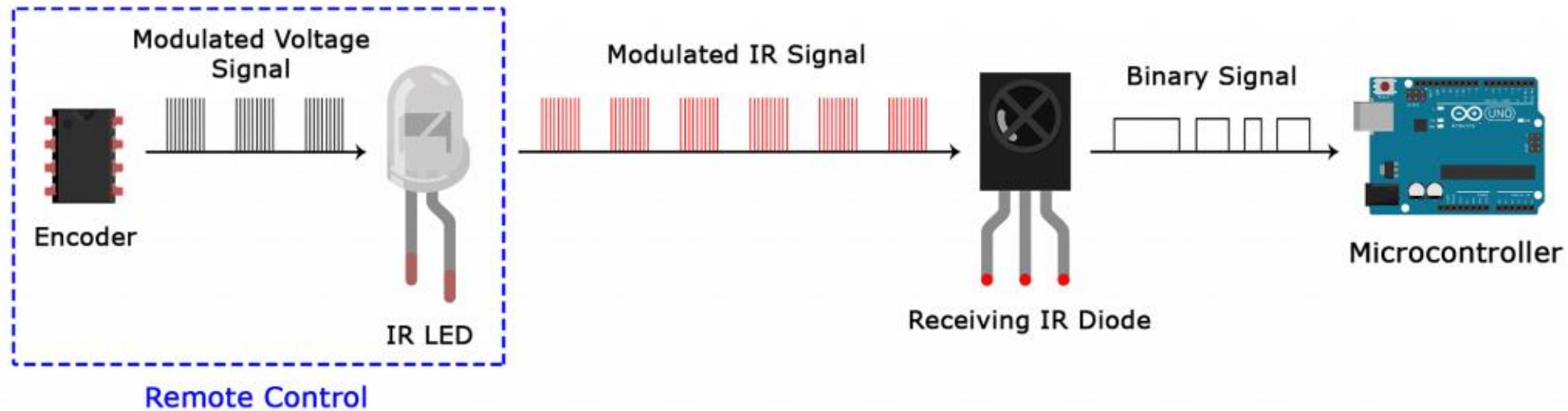
El infrarojo es una radiación electromagnética con longitud de onda mayor que la de la luz visible (frecuencia menor).

Longitud de onda: de 700 a 1000 micrómetros.

La radiación térmica de los objetos está en este rango, por ello se utiliza para construir cámaras térmicas que detectan la temperatura de los mismos.



Emisión - Recepción

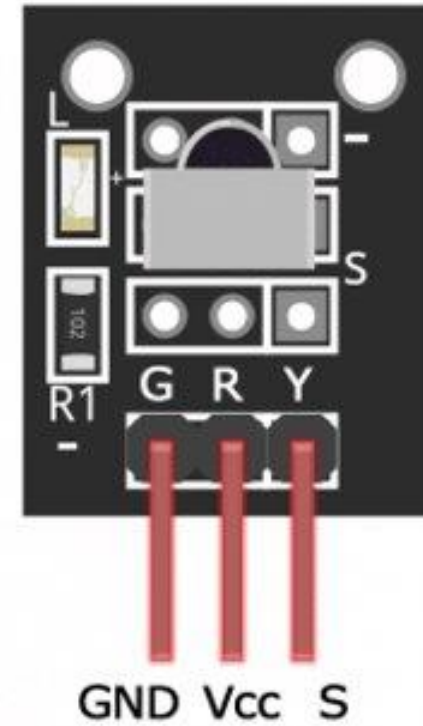
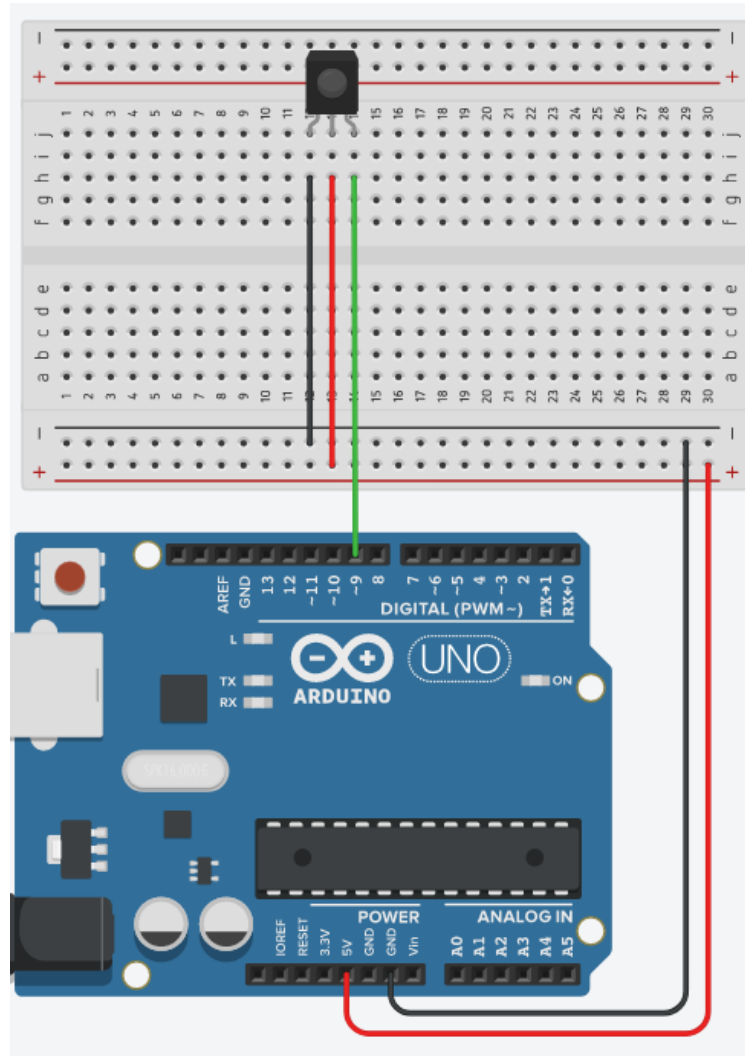


Sensor IR 950 nm + filtro

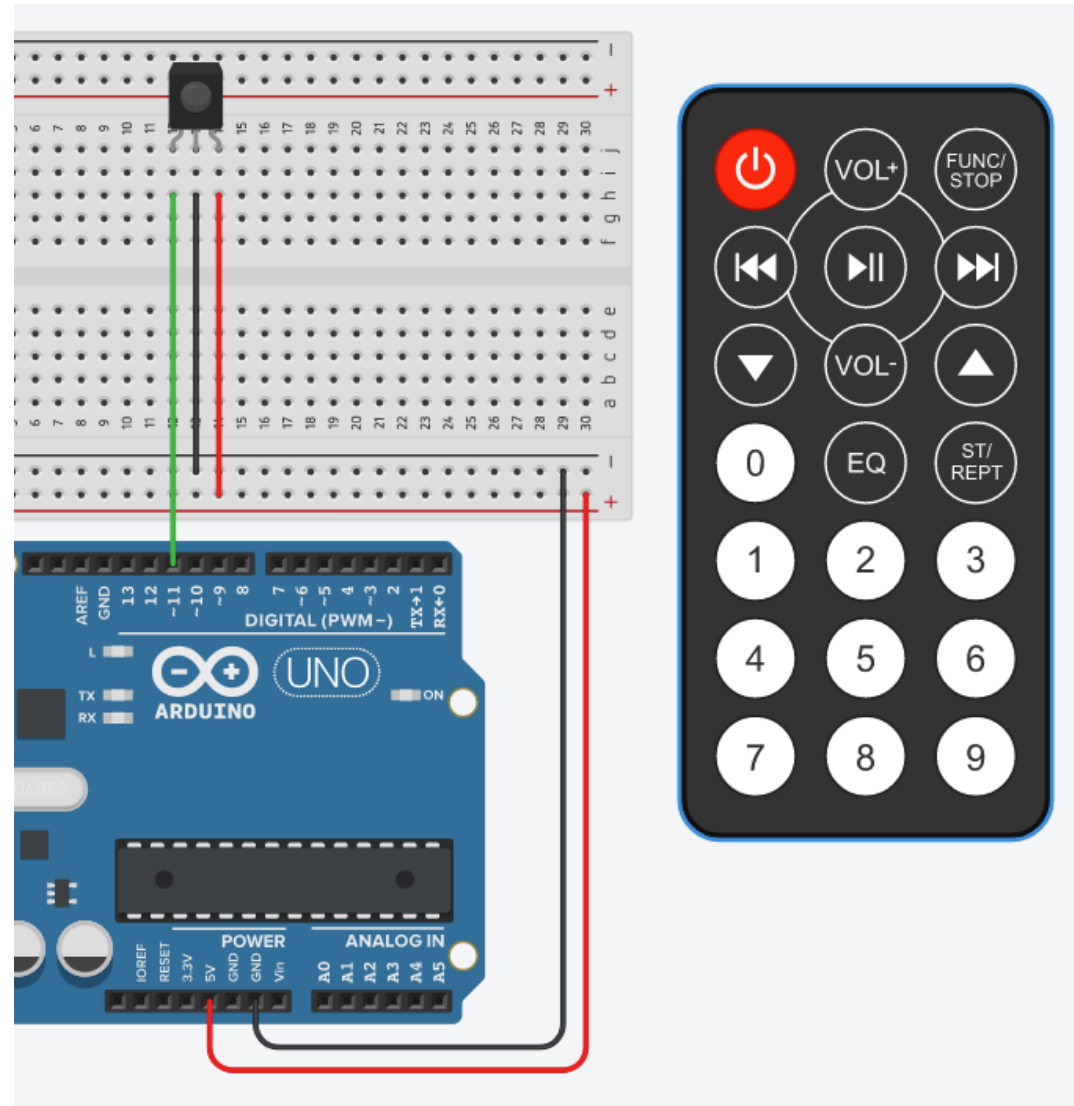
Actividad

1. Montar el circuito con el receptor IR. Es muy simple.
2. Cargar el código básico.
3. Obtener los valores hexadecimales asociados a cada tecla y anotarlos en el código como constantes.
4. Hacer la función translateIR() con los códigos numéricos (en hexadecimal) correctos
5. Montar el circuito a controlar:
 1. 4 LEDs
6. Programar:
 1. 4LEDs: Con las **teclas 1-4** encender y apagar cada LED. Con **power** apagar o encender todos.
XTRA: Con **PLAY** secuencia encendiendo un solo led cíclicamente cada segundo

Circuito Elegoo

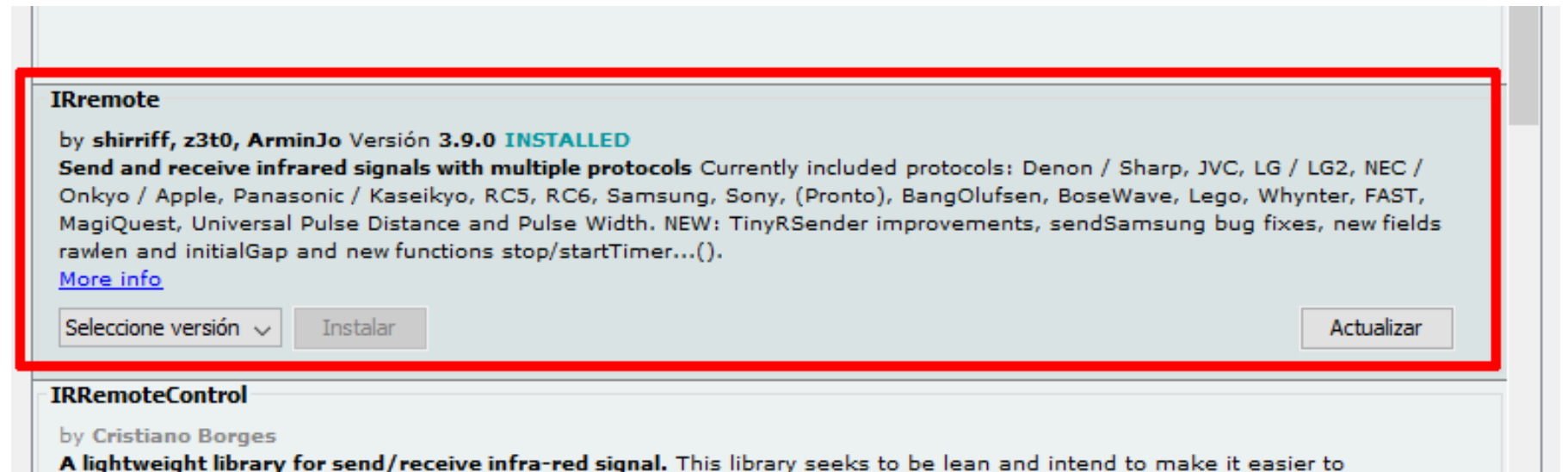


Circuito Tinkercad (para Testear)



Código: Librería

- Se necesita la librería **IRremote**. **Versión 3.9.0** para que coincida con la de Tinkercad
- En Tinkercad ya viene incluida, pero en Arduino IDE hay que instalarla.



Código

```
#include <IRremote.h>

const int RECV_PIN = 9; // Pin Data IR

void setup()
{
    Serial.begin(9600); // Serial
    // Arrancar recepción IR
    IrReceiver.begin(IR_RECEIVE_PIN, ENABLE_LED_FEEDBACK);
}

void loop()
{
    if (IrReceiver.decode()) { // Si hay nuevo dato IR
        Serial.println("++++ recibo IR ++++");
        long cmd = IrReceiver.decodedIRData.decodedRawData; // comando recibido
        Serial.print("Comando: ");
        Serial.println(cmd, HEX);
        // IrReceiver.printIRResultShort(&Serial); // Imprime mas informacion (opcional)
        // Aqui llamamos a una función para hacer algo según el código recibido
        comandoIR( cmd );
        IrReceiver.resume(); // Comenzar la recepción de otro mensaje IR
    }
}
```

Este código permite decodificar la señal infraroja recibida por el sensor.

Guarda el resultado en la variable results, y se recoge el resultado con results.value

Para ver su valor en formato hexadecimal se utiliza:

```
Serial.println(results.value, HEX);
```

Código: Anotar los códigos de cada tecla

```
const long KEY_POWER = 0xFF00FF;  
const long KEY_FSTOP = 0xFF00FF;  
const long KEY_VOLPLUS = 0xFF00FF;  
const long KEY_FBACK = 0xFF00FF;  
const long KEY_PAUSE = 0xFF00FF;  
const long KEY_FFWD = etc;  
const long KEY_DOWN = etc;  
const long KEY_VOLMIN = etc;  
const long KEY_UP = etc;  
const long KEY_EQ = etc;  
const long KEY_REPEAT = etc;  
const long KEY_1 = 0xFF00FF;  
const long KEY_2 = etc;  
const long KEY_3 = etc;  
const long KEY_4 = etc;  
const long KEY_5 = etc;  
const long KEY_6 = etc;  
const long KEY_7 = etc;  
const long KEY_8 = etc;  
const long KEY_9 = etc;  
const long KEY_0 = etc;
```

La transmisión de IR puede tener errores y no dar siempre un código correcto. Ignóralos!

Declarar constantes globales

Como parte de la actividad hay que encontrar los códigos hexadecimales de cada tecla que capta el receptor IR ya sea con el montaje físico como con el montaje virtual con Tinkercad.

Los códigos son diferentes en cada caso.

Para ello utilizar los mensajes escritos en el puerto Serial.

¡Ojo! Los códigos escritos en rojo no son correctos.

Código: Anotar los códigos de cada tecla

```
#define KEY_POWER = 0xFF00FF;
#define KEY_FSTOP = 0xFF00FF;
#define KEY_VOLPLUS = 0xFF00FF;
#define KEY_FBACK = 0xFF00FF;
#define KEY_PAUSE = 0xFF00FF;
#define KEY_FFWD = etc;
#define KEY_DOWN = etc;
#define KEY_VOLMIN = etc;
#define KEY_UP = etc;
#define KEY_EQ = etc;
#define KEY_REPEAT = etc;
#define KEY_1 = 0xFF00FF;
#define KEY_2 = etc;
#define KEY_3 = etc;
#define KEY_4 = etc;
#define KEY_5 = etc;
#define KEY_6 = etc;
#define KEY_7 = etc;
#define KEY_8 = etc;
#define KEY_9 = etc;
#define KEY_0 = etc;
```

Otra opción en lugar de utilizar variables tipo **const long** (en un int no caben esos números tan altos), es utilizar macros con **#define**.

Código: función comandoIR()

```
void comandoIR( long orden ) {  
    switch(orden) {  
        case KEY_POWER: Serial.println("POWER"); break;  
        case KEY_FSTOP: Serial.println("FUNC/STOP"); break;  
        case KEY_1: Serial.println("Tecla 1"); break;  
        case KEY_2: Serial.println("tecla 2"); break;  
        etc  
        ...  
  
        default:  
            Serial.println(" other button : ");  
            Serial.println(results.value);  
    } // End switch  
  
    delay(500); // Esperar un poco  
}
```

Con esta función decidimos qué hacer cuando se recibe un código u otro.

La función se puede llamar de cualquier forma!

¡Poned los códigos HEX!

Hay que incorporar los códigos de cada tecla y las acciones que se desee que el circuito haga.

Referencias

- Cap 13 Libro de Proyectos de Elegoo
- <https://www.circuitbasics.com/arduino-ir-remote-receiver-tutorial/>
- <https://www.luisllamas.es/arduino-mando-a-distancia-infrarrojo/>
- <https://tutorial45.com/arduino-ir-receiver/>

17

Motor DC

Transistor PNP y diodo

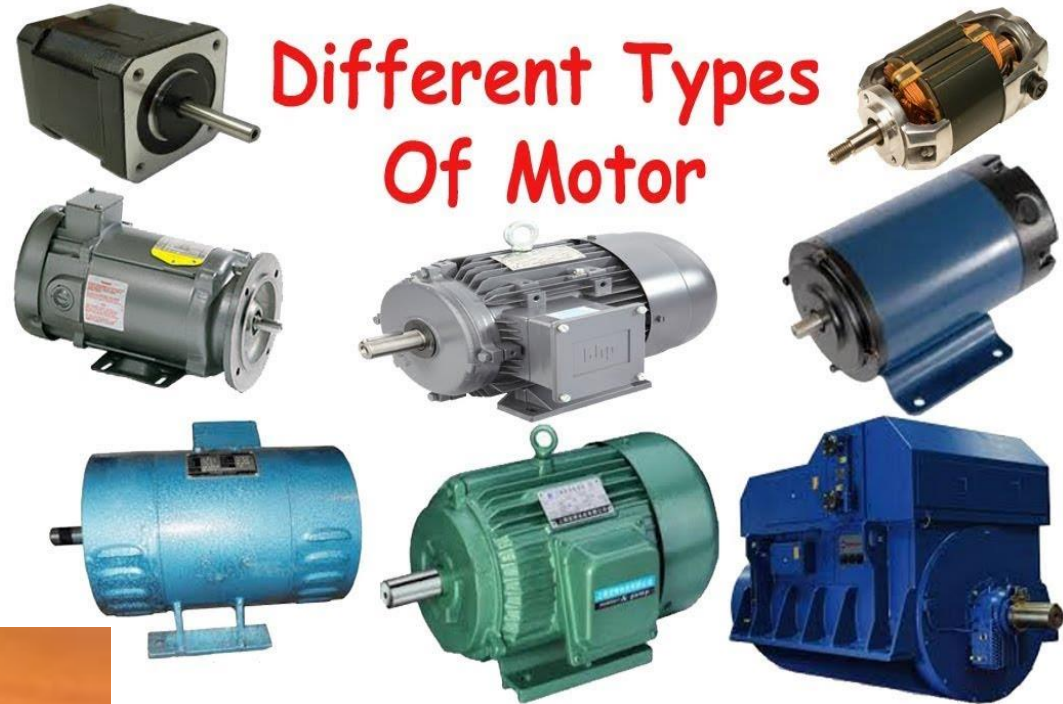
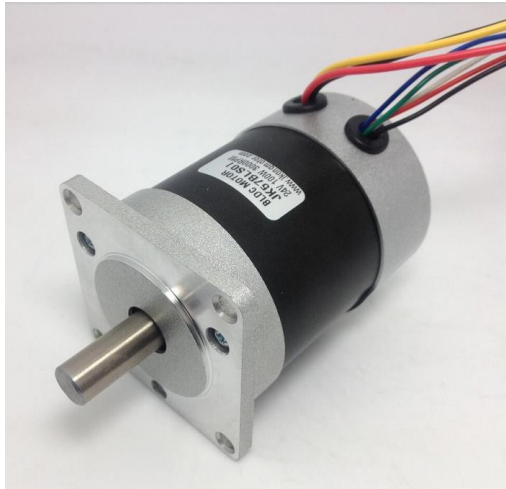
Cap 9 Libro de Proyectos de Arduino

Índice

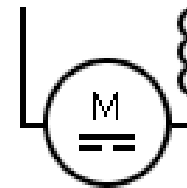
- Materiales:
 - Motor DC
 - Transistor NPN PN2222
 - Diodo rectificador
 - Batería
 - Pulsador
 - Potenciómetro
- Circuito
- Actividad: potenciómetro.

- **Motor:** Necesita más corriente que la que da Arduino (max 40mA en un pin digital).
- **Fuente de alimentación DC:** proporciona la energía necesaria para mover el motor. Puede ser una fuente diferente de la que alimenta el arduino. Hay que **poner en común** el negativo de las dos fuentes de alimentación (**masa**).
- **Transistor PNP:** amplifica la señal de Arduino para alimentar el motor independientemente de la energía que se aporta para que funcione.
- **Diodo Zener:** impide que la corriente inducida por el motor en movimiento (ppo. de acción-reacción) se dirija a la placa de arduino. Permite la corriente solo en una dirección.
- **Potenciómetro:** Controla la velocidad de giro del motor.

Motor DC



Different Types Of Motor

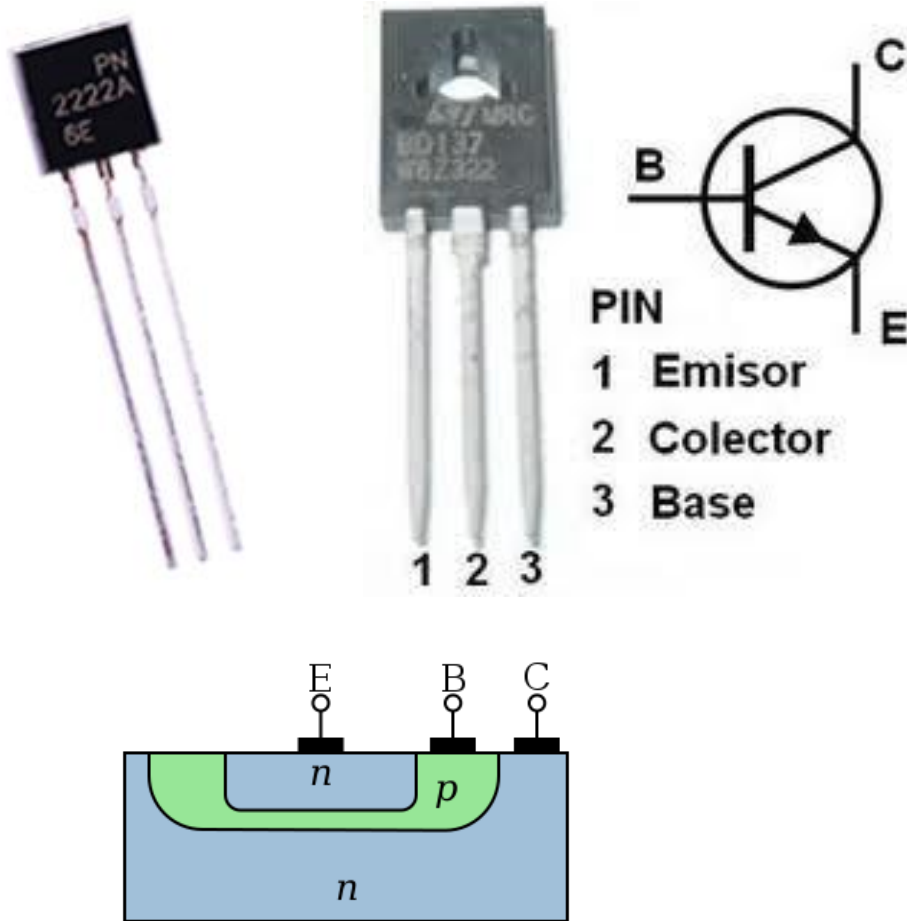


Características:

- Voltaje
- Potencia
- Velocidad de giro (rpm)

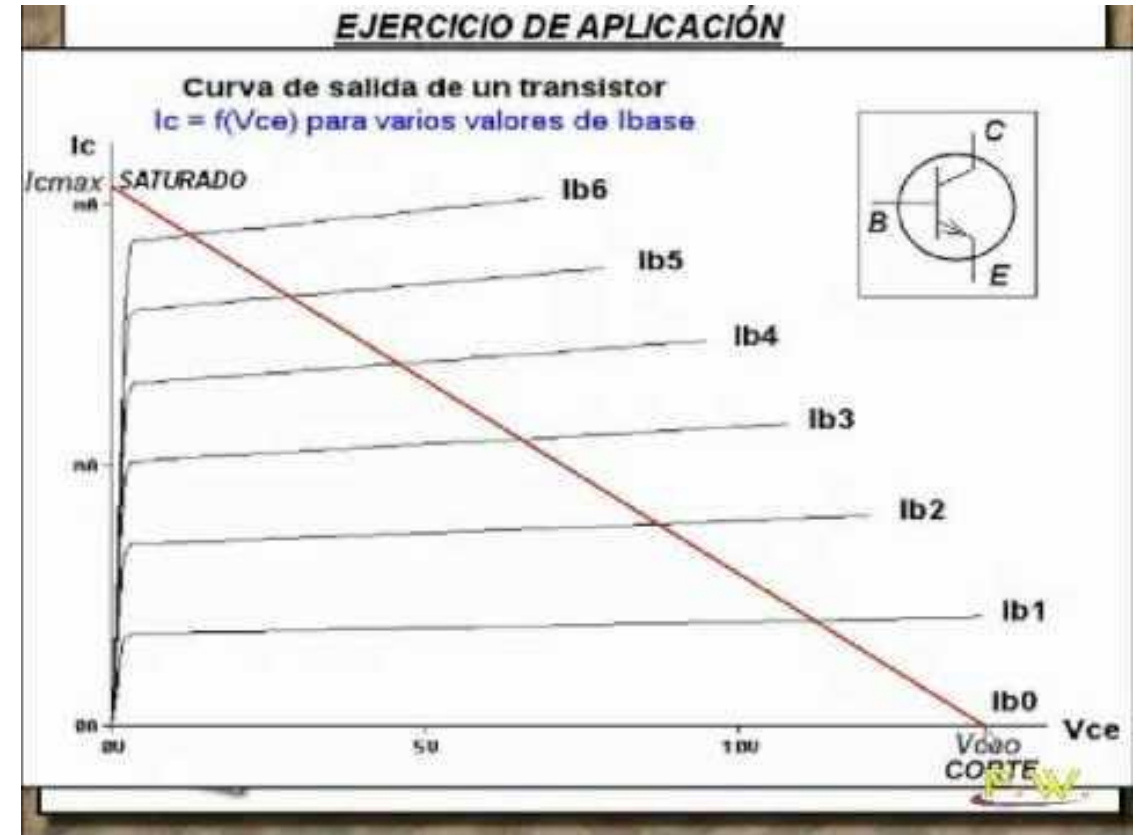
[Wiki](#)

Transistor NPN: PN2222



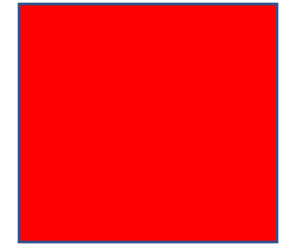
<https://circuitglobe.com/pnp-transistor.html>

https://en.wikipedia.org/wiki/Bipolar_junction_transistor

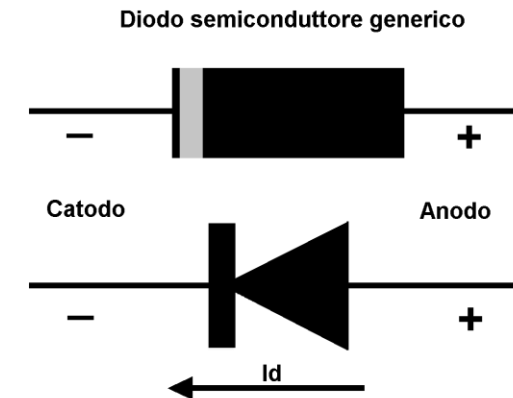
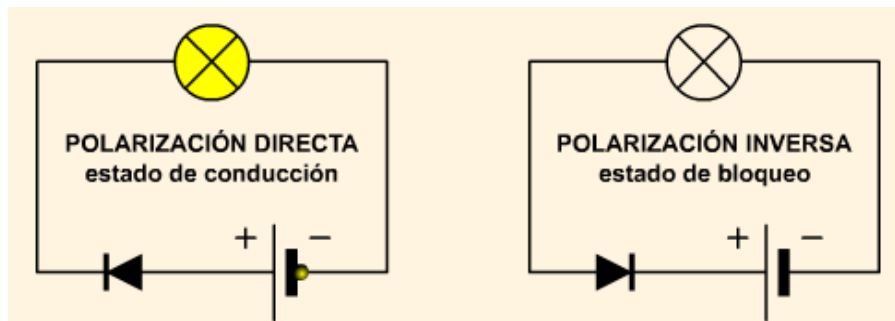


Controlando la pequeña corriente que metemos en la Base (I_b) controlamos el paso de una gran corriente desde el Colector al Emisor (I_c). **AMPLIFICACIÓN**

Conectar Diodo



- El diodo tiene **polaridad**. Hay que conectar un extremo (**cátodo**, el de la banda) a la parte negativa y el otro (**ánodo**) a la positiva.

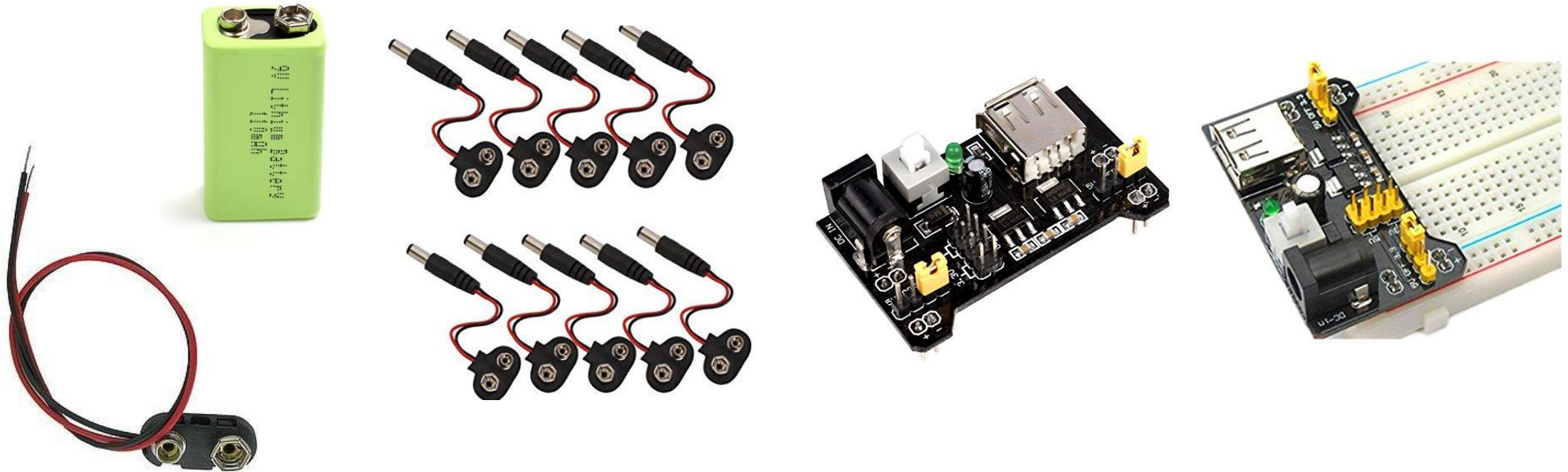


[Como conectar un diodo](#)

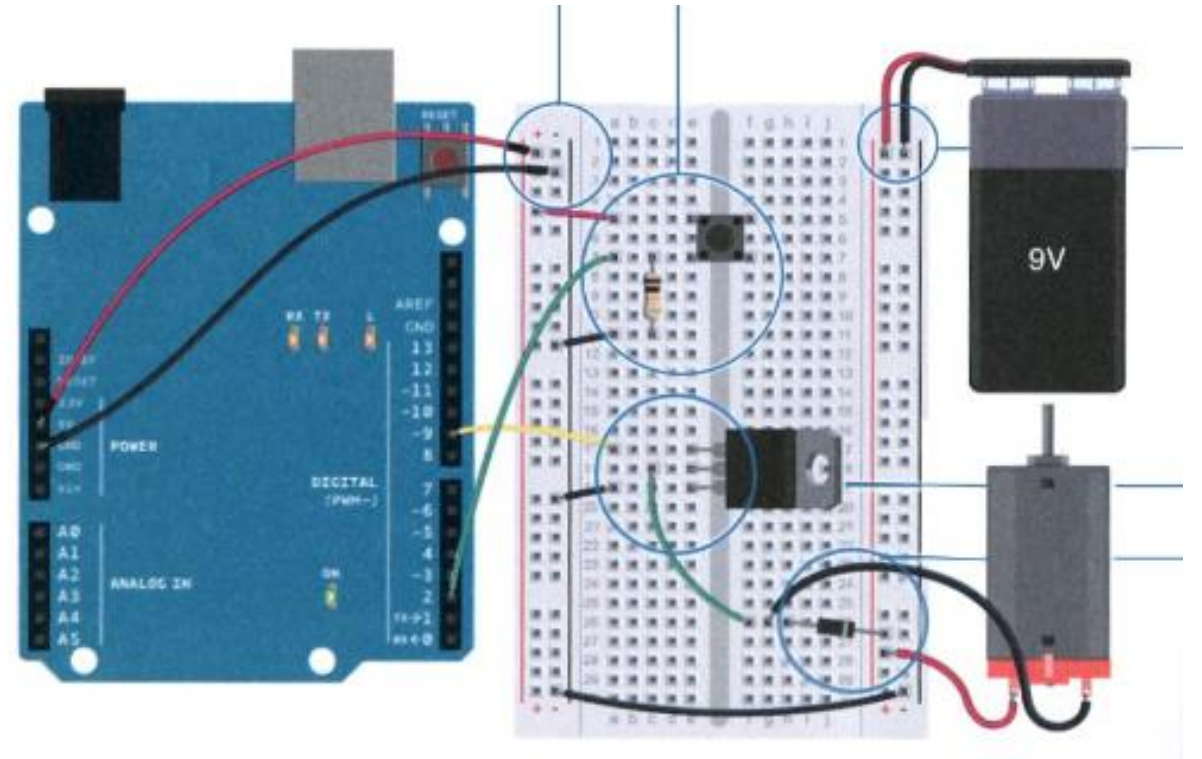
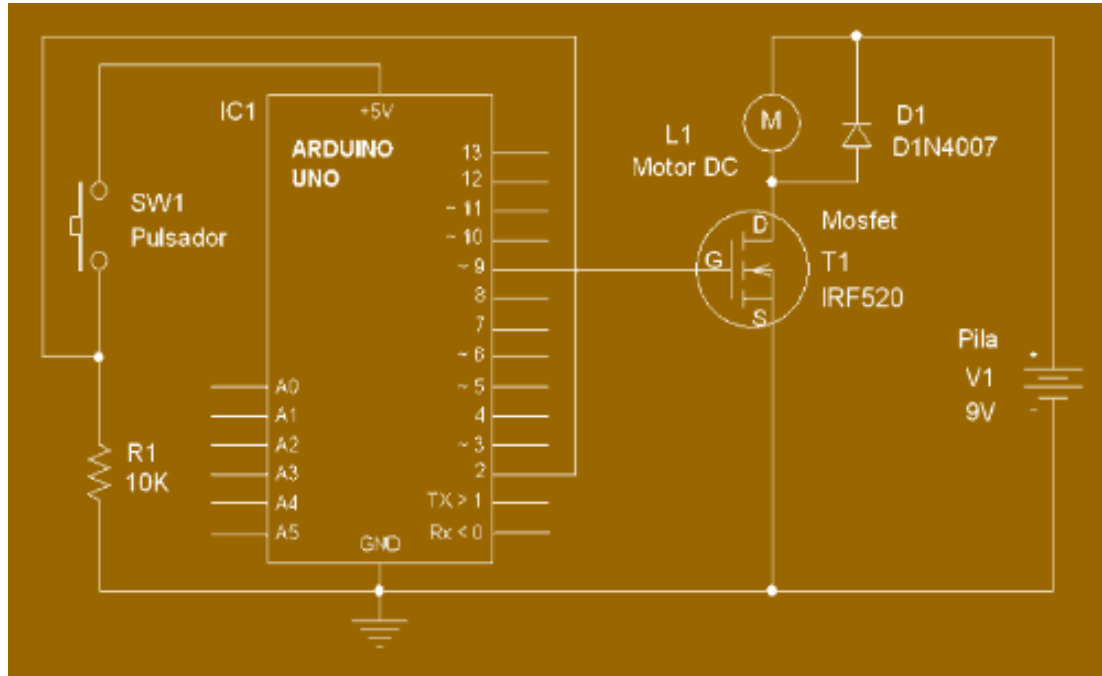
[Diodo Zener](#)

Fuente Alimentación

Utilizamos una **pila de 9V**. Se podría utilizar otra fuente de alimentación que de un voltaje similar
El motor DC de Elegoo es de 6-9 V. Se puede asegurar que no nos pasamos del voltaje con el **transformador** que viene incluido en el pack.

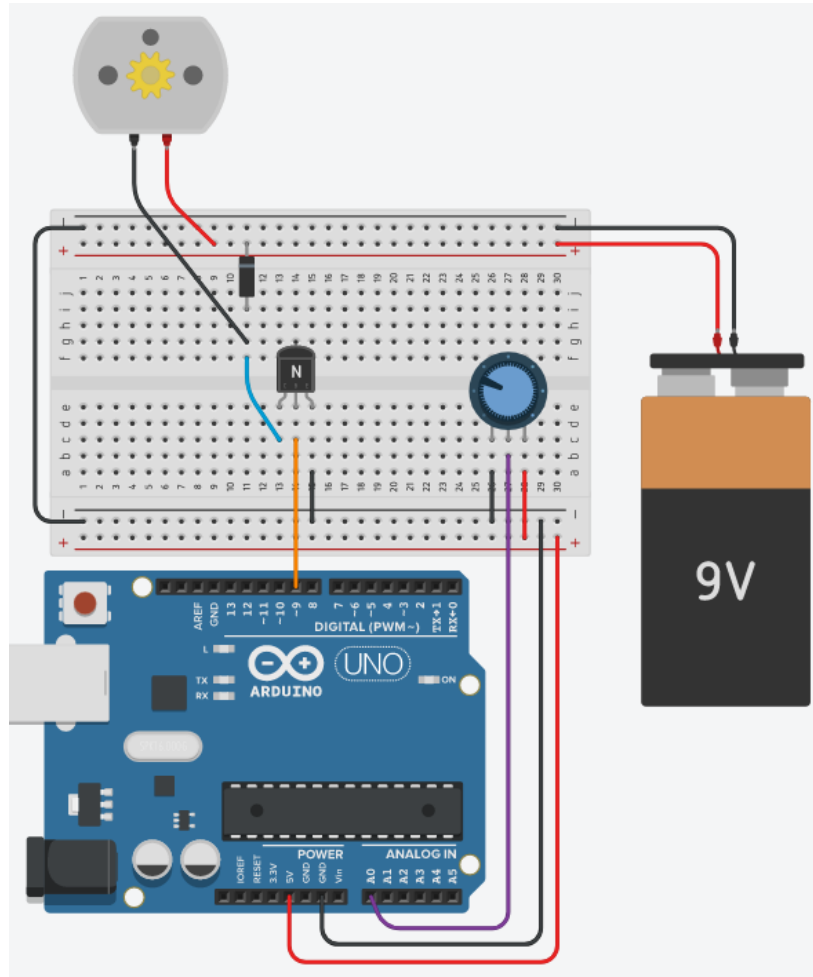


Circuito

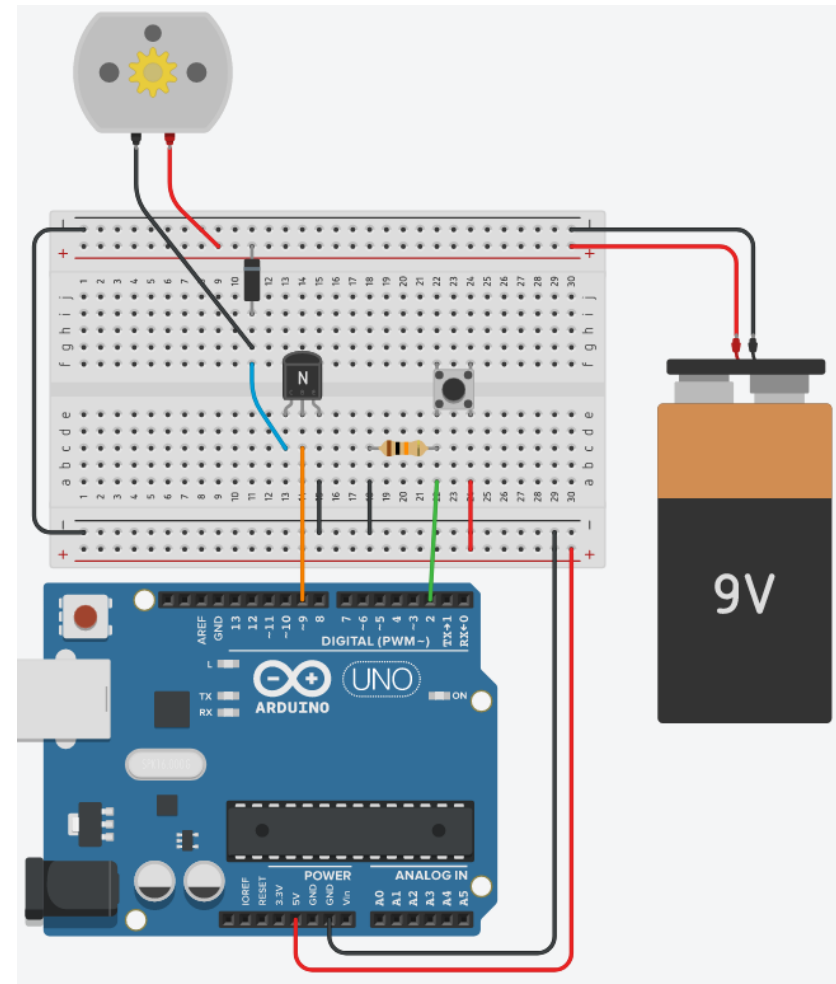


Circuito

Con potenciómetro



Con pulsador

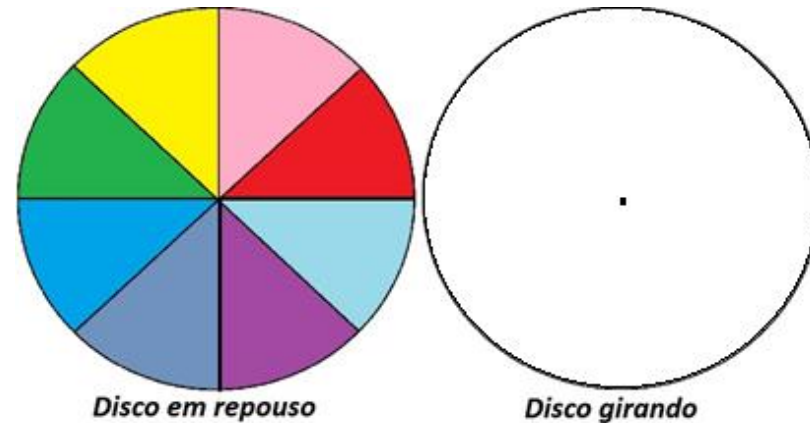


Actividad

- Montar el circuito de motor DC con el potenciómetro
- Programar el circuito para poder controlar la velocidad de giro del motor con el potenciómetro.
- Incorporar un pulsador para que solo gire si se mantiene pulsado (o al revés, que no gire si se pulsa)

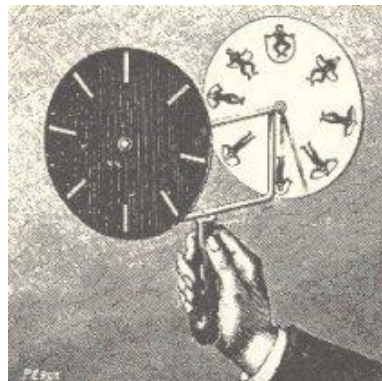
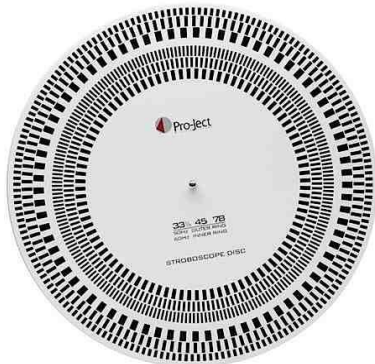
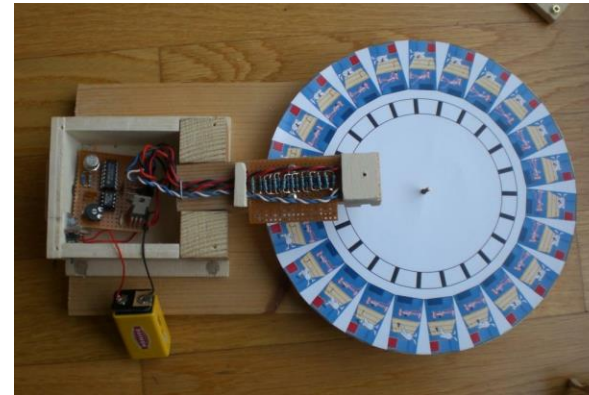
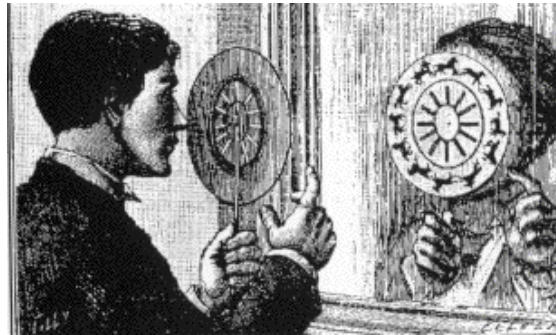
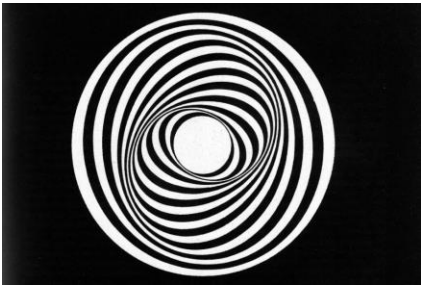
Sugerencia

- **Disco de Newton:** Preparar un disco con varios colores y poner en una superficie circular rígida acoplada al motor. Se observarán mezclas de colores, e incluso el blanco, dependiendo de los colores utilizados.



Sugerencia

- **Disco estroboscópico:** Imprimir un disco con marcas repetidas a distintos intervalos dependiendo del radio donde estén las marcas. Comprobar qué ocurre con los patrones cuando el disco gira a una velocidad u otra.
- También se puede utilizar con animaciones impresas en el disco. Se necesita hacer una ventanita para ver dentro de ella la animación.



[Enlace Búsqueda disco estroboscópico](#)



Sugerencia

- **Zoótropeo**: Disco giratorio con animaciones impresas, o con figuras 3D. Se combina con una ventanita o con luces estroboscópicas para crear una impresión de animación fijando la posición del sico que se desea que se imprima en el ojo.



["All Things Falls" Vimeo](#) (enlace)

[Enlace: La masacre de los inocentes](#)
(Vice Magazine)



[4-Motion 3D](#)
[zoetropes](#) (enlace)

Referencias

- Cap 09 Libro de Proyectos de Arduino
- Vídeo youtube: <https://www.youtube.com/watch?v=32bj9VXDiCQ>
- <https://mecabot-ula.org/tutoriales/arduino/practica-9-control-de-un-motor-de-corriente-continua/>

18

Motor DC

Controlador L293D (Puerta H)

Cap 10 Libro de Proyectos de Arduino

Cap 21 Libro de Proyectos de Elegoo

Índice

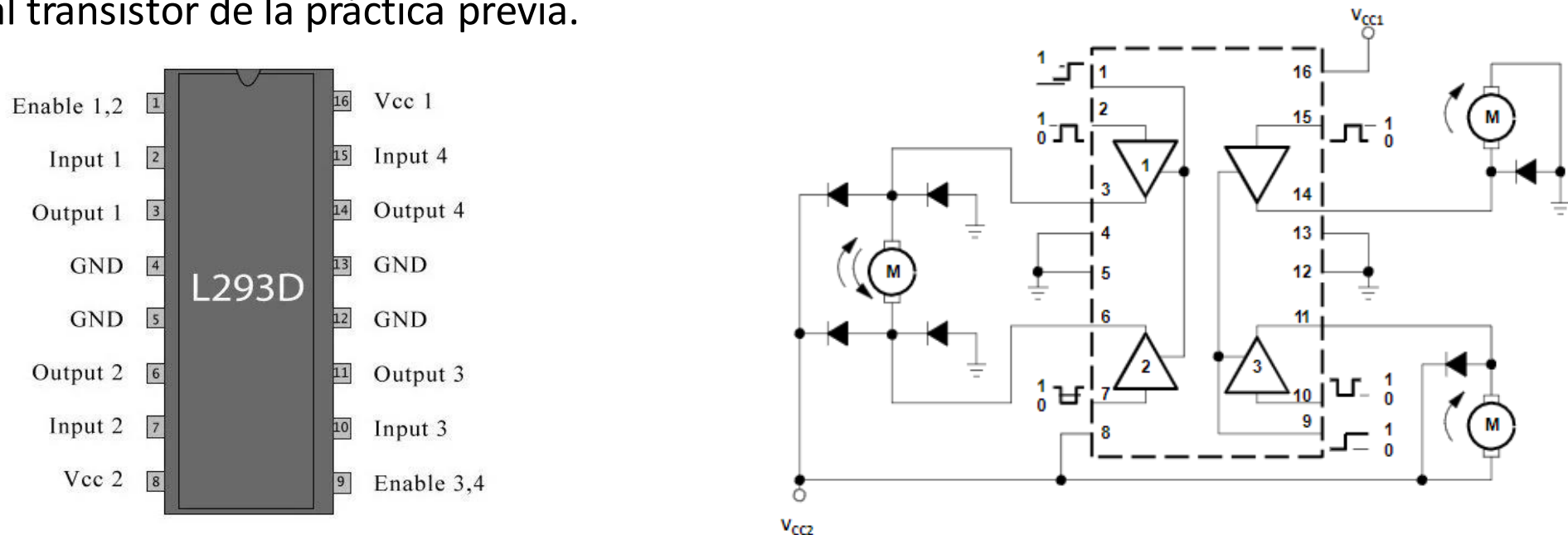
- Materiales:
 - Motor DC
 - Puerta H: controlador L293D
 - Batería 9V
 - 2 x Pulsador
 - Potenciómetro
 - Módulo de alimentación
- Circuito
- Actividad

Componentes

- **Puente-H L293D:**

Un puente-H es un circuito integrado (CI). En el interior de la cápsula tienen un circuito mucho mayor. Se controla conectando las patas metálicas a otros circuitos. Están protegidos por una cápsula de características diferentes dependiendo de la finalidad del circuito. Existen multitud de tipos para diferentes funcionalidades.

En este caso el **L293D** permite controlar dos motores DC. En su interior hay amplificadores, similares al transistor de la práctica previa.



Componentes

- **Control del L293D**

Cada lado permite controlar un motor DC.

ALIMENTACIÓN

Pin 16: Vcc1 de la pila/batería para alimentar el CI. 5V

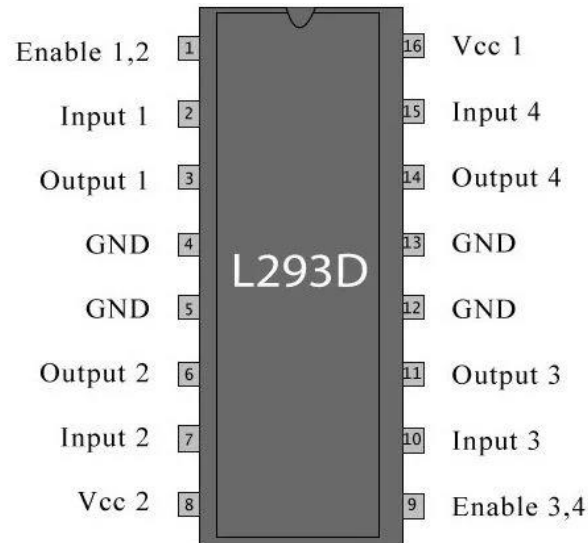
Pin 8: Vcc2 de la pila/batería para alimentar el motor. De 4.5 - 36V

Pin 1: Activa o desactiva el **motor L**. Se conecta al PWM para controlar la velocidad de giro del motor.

Pin 2, Pin 7: Indican el sentido de giro del motor: deben estar uno en LOW y otro en HIGH para girar. Si se intercambian los valores cambia el sentido de giro. Si los valores son iguales no hay giro.

Pines 4, 5: Tierra.

Pines 3, 6: Salida a los bornes del motor DC.



Pin 9: Activa o desactiva el **motor R**. Se conecta al PWM para controlar la velocidad de giro del motor.

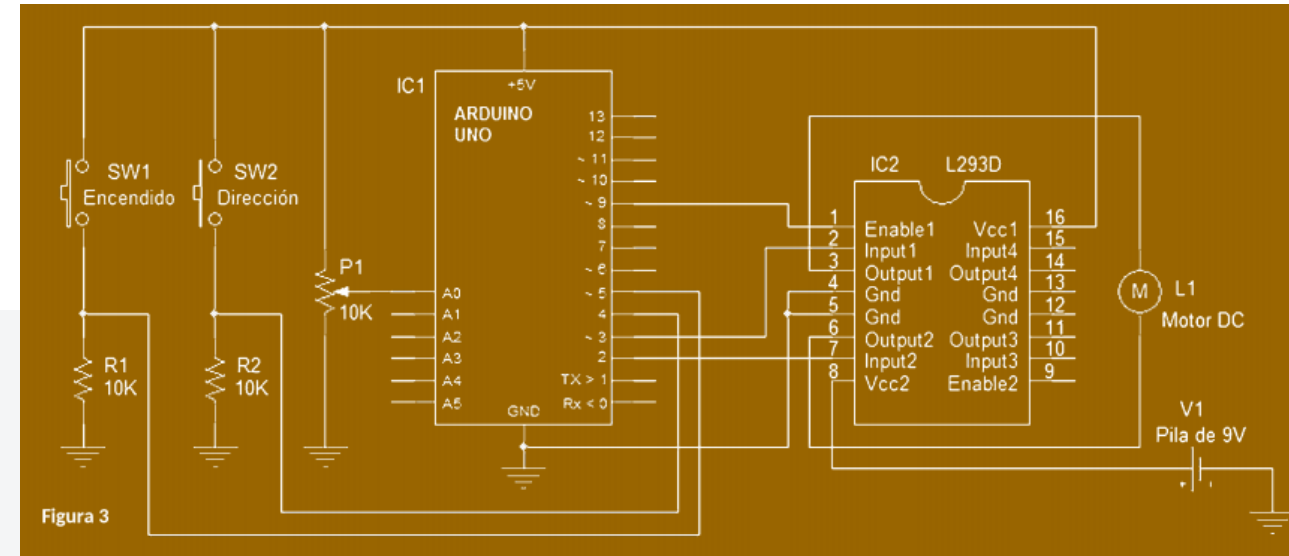
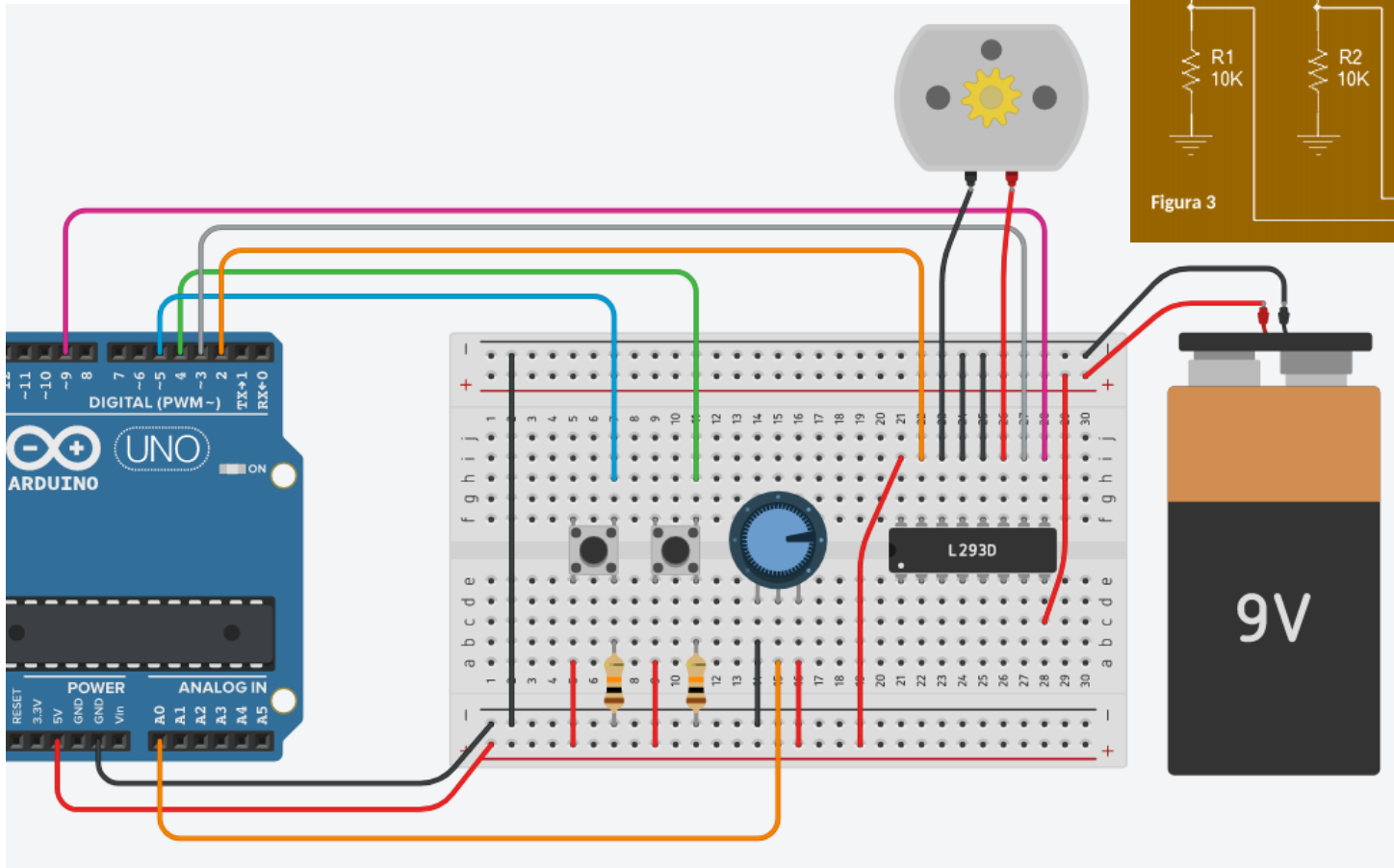
Pin 10, Pin 15: Indican el sentido de giro del motor: deben estar uno en LOW y otro en HIGH para girar. Si se intercambian los valores cambia el sentido de giro. Si los valores son iguales no hay giro.

Pines 12, 13: Tierra.

Pines 11, 14: Salida a los bornes del motor DC.



Circuito



Programación

- Utilizaremos varios pines digitales para controlar el motor:
 - **1 Pin para Activar el Motor:**
 - Debe ser **PWM** para poder controlar la **velocidad de giro** del motor
 - **2 Pines de control de giro:**
 - Tienen que estar uno en **Low** y otro en **High** para definir una dirección de giro. Se invierte la dirección si se intercambian los valores

Actividad

- Montar el circuito:
 - Primero: la parte relativa al controlador, motor, batería...
 - Se puede utilizar el conversor de voltaje que viene con Elegoo
 - Segundo:
 - 2 pulsadores + potenciómetro
- Seguir los pasos para la programación:
 - Examinar bien las características de control del controlador L2930D.
 - Programar:
 - un pulsador on/off
 - El potenciómetro controla la velocidad de giro
 - Un pulsador para cambiar de dirección
 - **XTRA:** ¿Cómo quitamos el pulsador de cambio de giro y controlamos la dirección con el potenciómetro?

Referencias

- <https://lastminuteengineers.com/l293d-dc-motor-arduino-tutorial/>
- **L293D**. Características y montaje con condensadores. <https://www.instructables.com/id/L293D-Motor-Driver/>
- Arduino y motores DC con L293D y L298D. <https://aprendiendoarduino.wordpress.com/2017/06/24/motores-arduino/>

19

Motor Paso a Paso (Stepper Motor)

Caps 23 y 24 Libro de Proyectos de Elegoo

Índice

- Materiales:
 - Motor Paso a Paso: **Stepper motor 28BYJ-48**
 - Driver **ULN2003A** para el Stepper
 - Cables M-H
 - Batería 9V o fuente de alimentación de 5V (cargador de móvil, por ejemplo)
 - Módulo de alimentación Elegoo
- Circuito
- Actividad
- Referencias

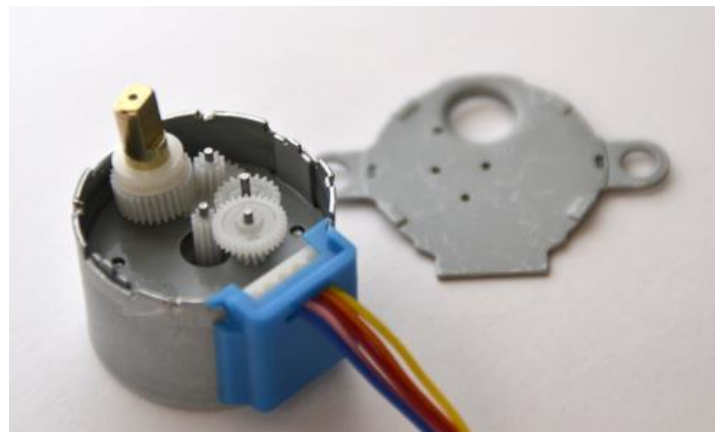
Componentes

Stepper Motor:

Es un dispositivo electromecánico que transforma pulsos de corriente en movimientos mecánicos discretos (pasos).

La velocidad de rotación depende de la frecuencia con la que se envían los pulsos eléctricos. El ángulo de rotación depende del número de pulsos recibidos. Esto da un gran control de la posición en mecanismos controlados por este tipo de motores, sin necesidad de sensores para controlar la posición (open loop system).

Se usa mucho en **impresoras 3D**, **brazos robóticos**...

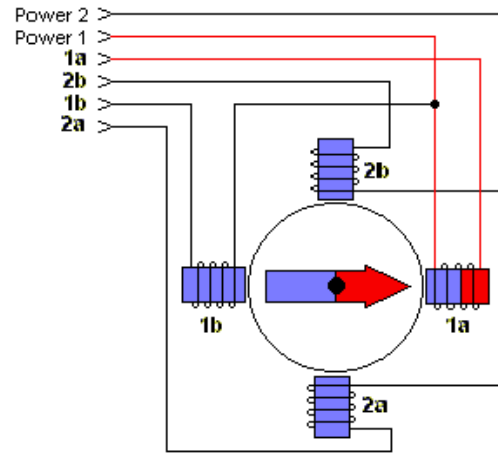


Componentes

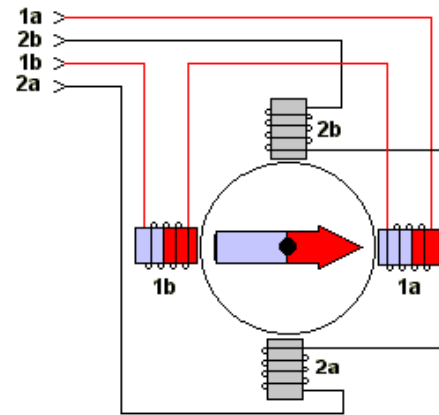
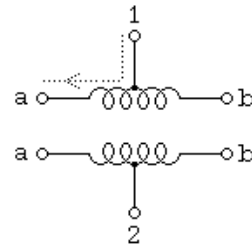
Stepper Motor:

Simplificando, internamente el stepper consiste en una pieza central imantada rodeada de un cierto número de bobinas. Según sea el núcleo central y la disposición de las bobinas hay dos tipos: unipolares y bipolares.

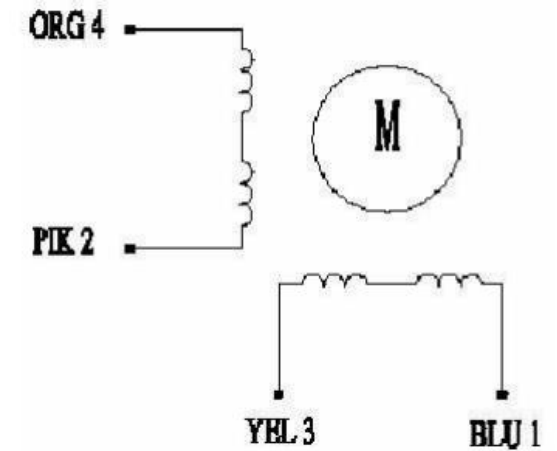
Para que rote hay que activar en orden uno de los diferentes bobinados manteniendo apagados el resto y así avanza un paso, después se enciende la siguiente bobina y se apaga el resto, y así sucesivamente...



Conceptual Model of Unipolar Stepper Motor



Conceptual Model of Bipolar Stepper Motor



Componentes

- **Stepper Motor 28BYJ-48 - Características**

Unipolar de 5 cables (4 bobinas)

Funciona a 5V o 12V

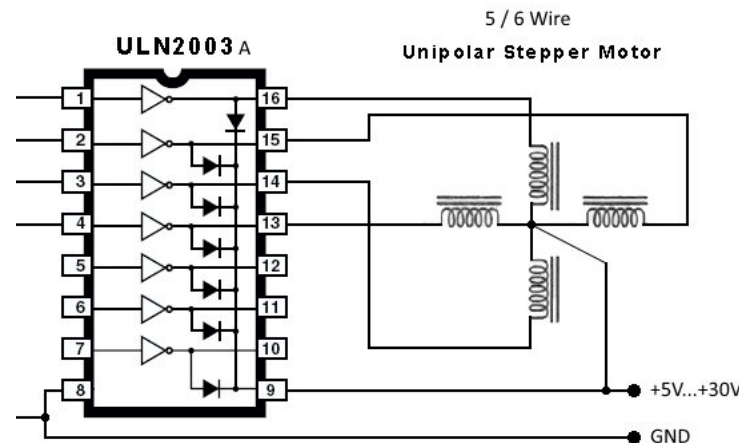
Paso de 1/64 de vuelta = 5.625°

Caja reductora de 1/64 $\Rightarrow$ Paso de $5.625/64 = 0.08789^\circ$

Rpm: 0-17ç

Torque de 34 Newton Metro más o menos 35 gramos por cm

Frecuencia máxima 100Hz que equivale a un delay de 10 ms



Stepper motor 28BYJ-48 Parameters

- Model: 28BYJ-48
- Rated voltage: 5VDC
- Number of Phase: 4
- Speed Variation Ratio: 1/64
- Stride Angle: $5.625^\circ / 64$
- Frequency: 100Hz
- DC resistance: $50\Omega \pm 7\%$ (25°C)
- Idle In-traction Frequency: $> 600\text{Hz}$
- Idle Out-traction Frequency: $> 1000\text{Hz}$
- In-traction Torque $> 34.3\text{mN.m}$ (120Hz)
- Self-positioning Torque $> 34.3\text{mN.m}$
- Friction torque: 600-1200 gf.cm
- Pull in torque: 300 gf.cm
- Insulated resistance $> 10\text{M}\Omega$ (500V)
- Insulated electricity power: 600VAC/1mA/1s
- Insulation grade: A
- Rise in Temperature $< 40\text{K}$ (120Hz)
- Noise $< 35\text{dB}$ (120Hz, No load, 10cm)

Componentes

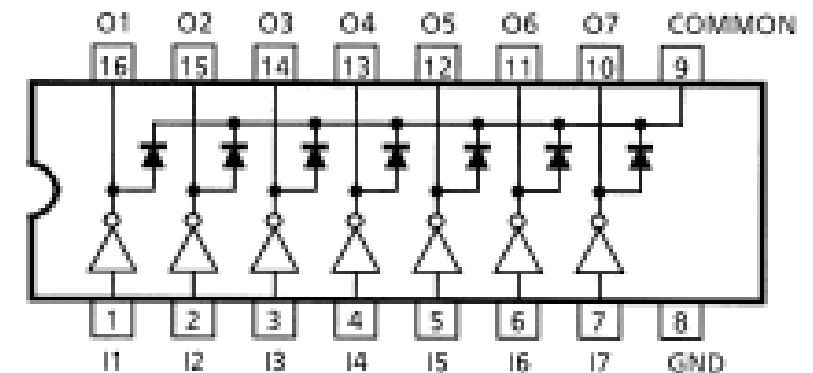
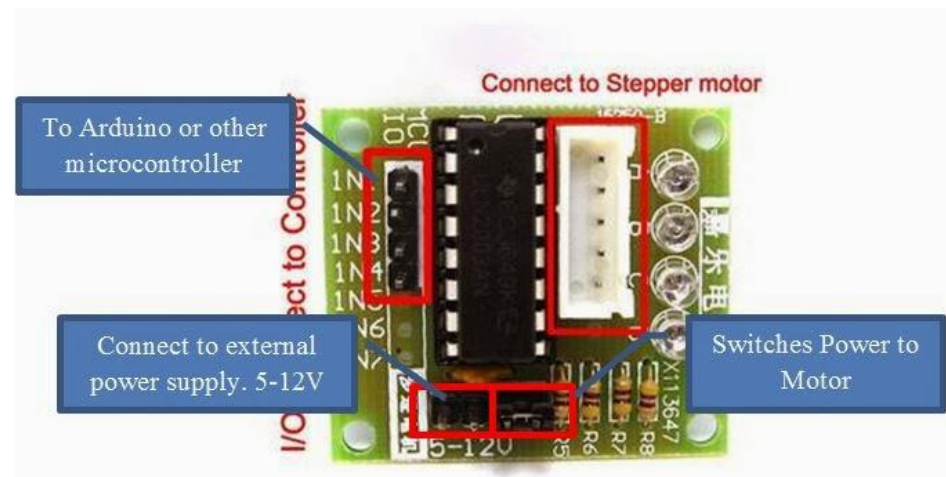
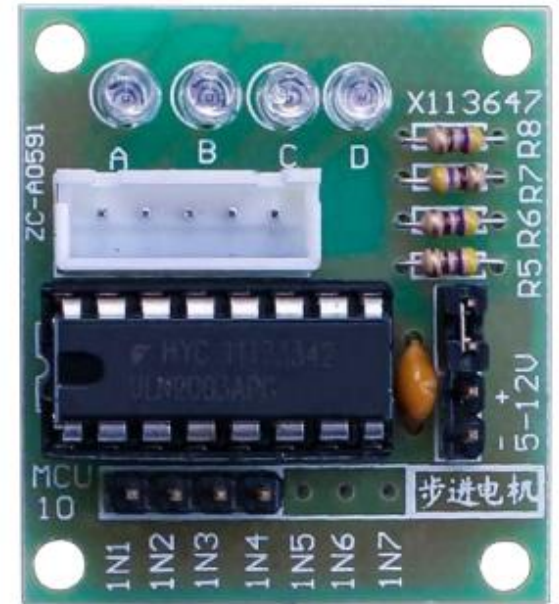
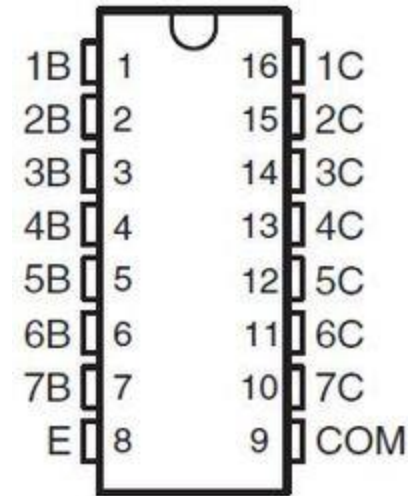
- **Driver ULN2003A**

Contiene **7 transistores NPN Darlington**, que permiten una gran amplificación de corriente.

Un par Darlington son dos transistores NPN conectados consecutivamente, de forma que el segundo amplifica lo que ya amplifica el primero.

El driver ULN2003A permite corriente de hasta **500mA** y voltajes de **50V**.

Se utilizan habitualmente para controlar relés, motores paso a paso, pantallas de LEDs...



Funcionamiento stepper Motor

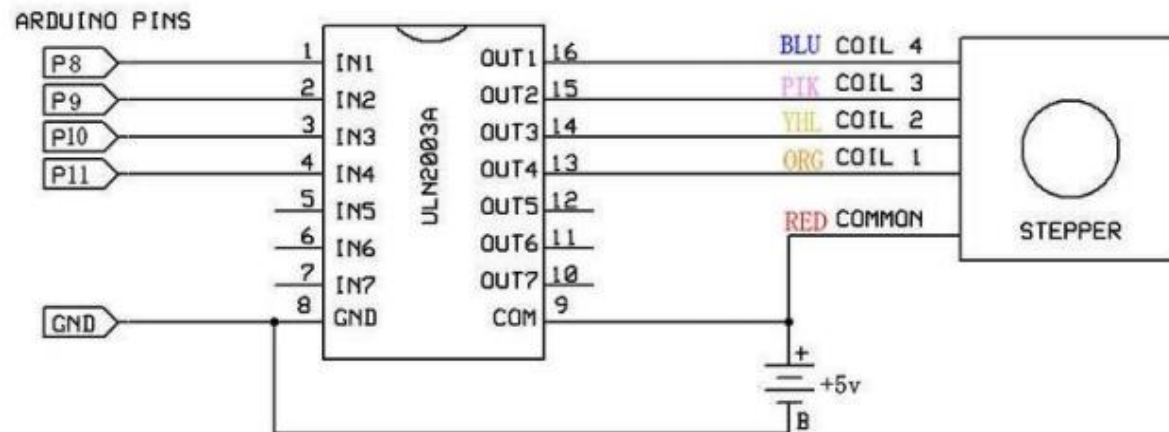
[Animación funcionamiento de stepper motor](#)

Lead Wire Color	---> CW Direction (1-2 Phase)							
	1	2	3	4	5	6	7	8
4 ORG	-	-						-
3 YEL		-	-	-				
2 PIK				-	-	-		
1 BLU						-	-	-

Here are schematics showing how to interface a unipolar stepper motor to four controller pins using a ULN2003A, and showing how to interface using four com

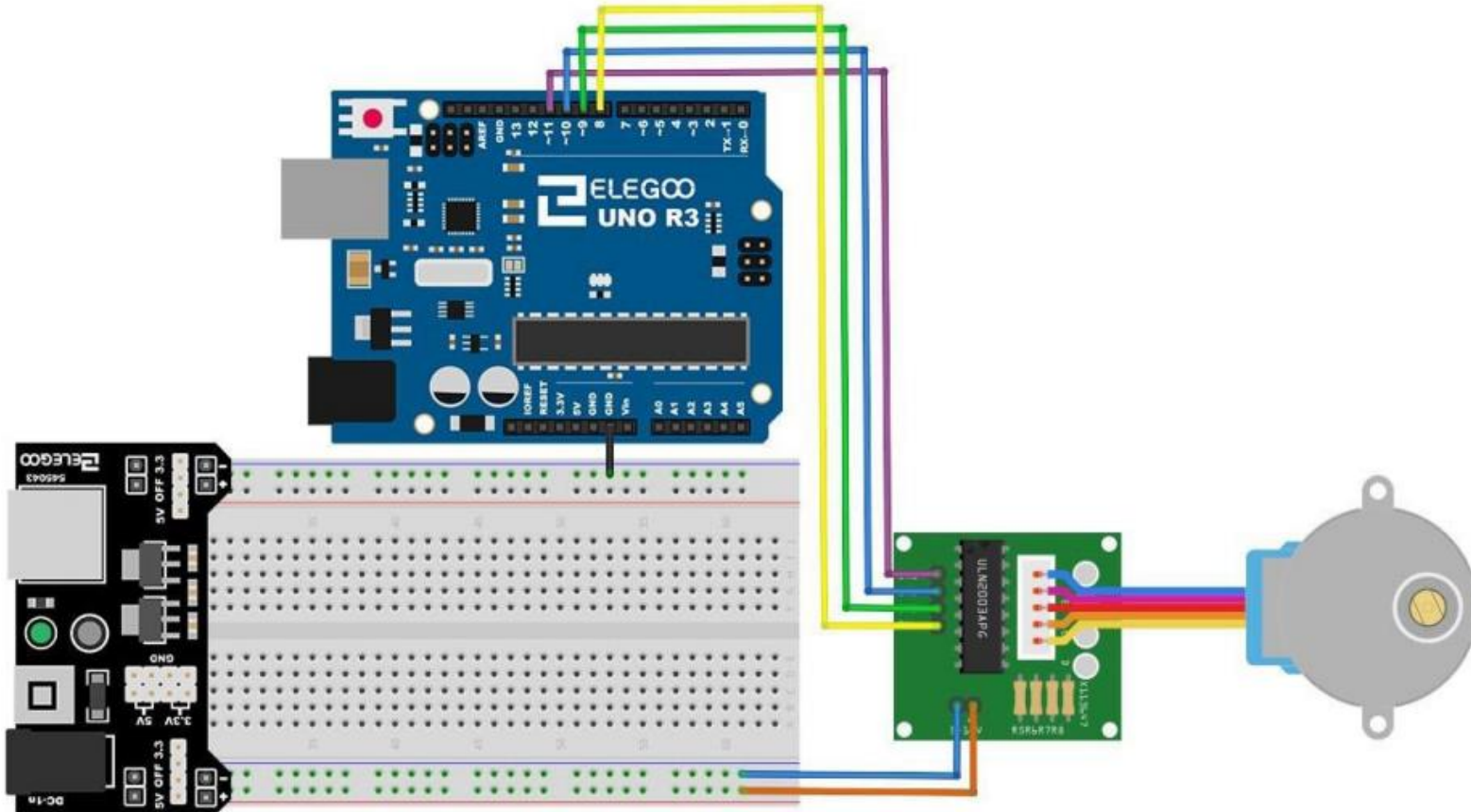
Hay varios esquemas diferentes de funcionamiento del motor.

Cada uno tiene distintas fases en la que se encienden y apagan unas u otras bobinas.



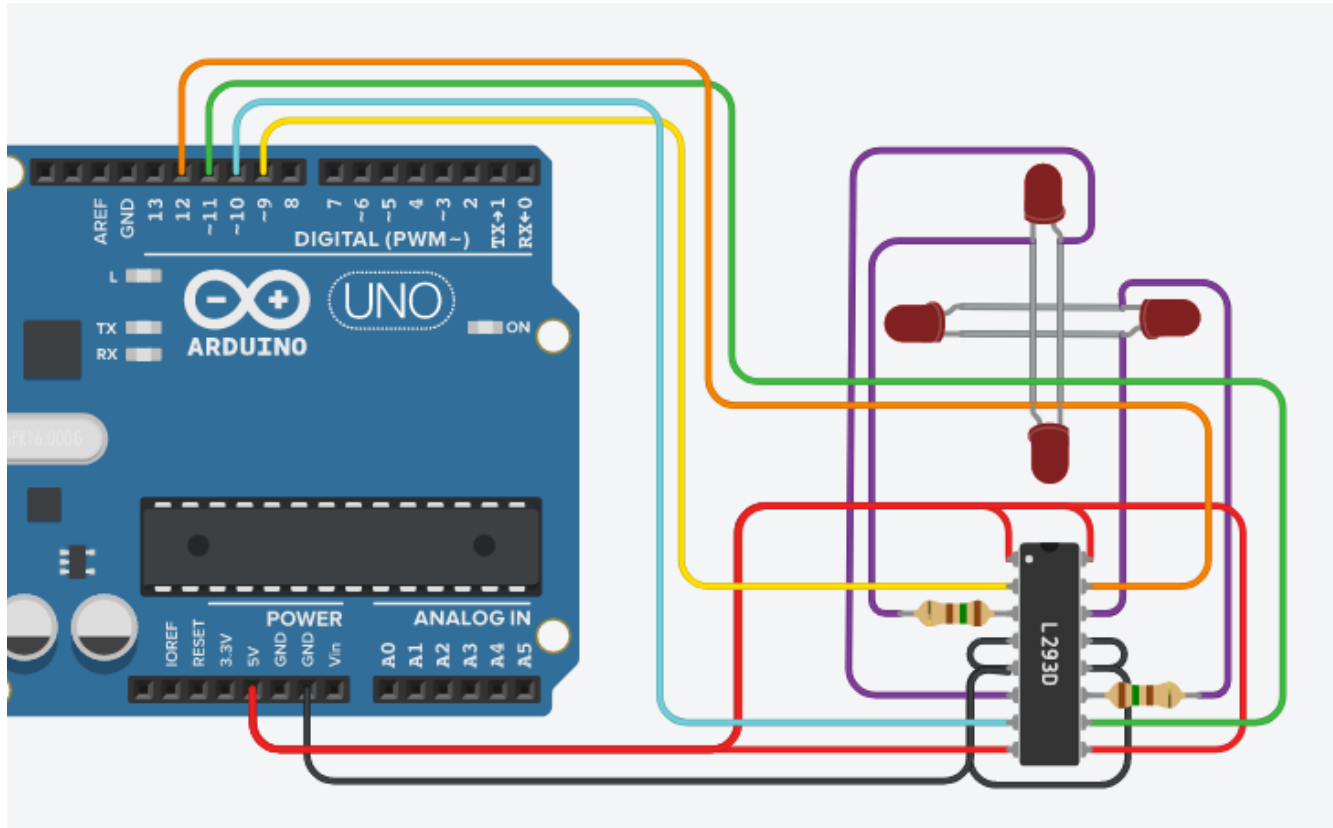
Circuito

Poner ambos
jumpers a 5V



Circuito Tinkercad

En Tinkercad no hay motor paso a paso ni controlador ULN2003A. Nos limitamos a simular como se envían los impulsos desde arduino.



El código de este montaje mostrará como se activan y desactivan las bobinas que mueven la pieza central.

Actividad. (No hay entrega de esta práctica)

- Solo se puede hacer con Arduino. Para tinkercad mostro un montaje alternativo que enciende LEDs
- Montar el sencillo circuito.
- Poner un cartón o similar en el potenciómetro para apreciar visualmente el giro.
- Montar un potenciómetro (u otro sensor) al A0 para regular la velocidad y la dirección de giro
- Programación A: Sin librería
- Programación B: Con librería Stepper
 - Incluir un pulsador para que haga un movimiento
 - Incluir un potenciómetro para controlar la cantidad de movimiento
 - Incluir un pulsador para cambiar de dirección

Código: sin librería

Se activan y desactivan de forma manual las bobinas para hacer girar el imán central.

1) Configurar los pines necesarios como output

2) Incluir un delay de unos 10ms

```
digitalWrite(motorPin1, HIGH);  
digitalWrite(motorPin2, LOW);  
digitalWrite(motorPin3, LOW);  
digitalWrite(motorPin4, LOW);  
delay(delayTime);
```

```
digitalWrite(motorPin1, LOW);  
digitalWrite(motorPin2, HIGH);  
digitalWrite(motorPin3, LOW);  
digitalWrite(motorPin4, LOW);  
delay(delayTime);
```

```
digitalWrite(motorPin1, LOW);  
digitalWrite(motorPin2, LOW);  
digitalWrite(motorPin3, HIGH);  
digitalWrite(motorPin4, LOW);  
delay(delayTime);
```

```
digitalWrite(motorPin1, LOW);  
digitalWrite(motorPin2, LOW);  
digitalWrite(motorPin3, LOW);  
digitalWrite(motorPin4, HIGH);  
delay(delayTime);
```

Código: con librería

Librería <Stepper.h>

Hay ejemplos en el IDE de Arduino

Hay que indicar los pines a los que esta conectado.

Hay que indicar el **número de pasos necesarios** para dar una vuelta completa. Es característico del motor. En nuestro caso es **2048**.

Hay que indicar una **velocidad de rotación (en rpm)** adecuada al motor. En nuestro motor puede ser hasta unas 17rpm. Mejor utilizar **3-10 rpm**.

```
int stepsPerRevolution = 2048;
int motorSpeed = 5;
int dt = 500; // No es necesario.
Stepper miMotor(stepsPerRevolution, 8,10,9,11); //IMPORTANTE EL ORDEN DE LOS PINes
```

En loop:

```
miMotor.setSpeed(motorSpeed); // Esto también se puede fijar en setup()
int step = 200; // Indicamos el numero de pasos
miMotor.step(step); // El step puede ser negativo
delay(dt);
```

Referencias

- Arduino Tutorial 35: Understanding How to Use a Stepper Motor – Con kit de Elegoo - <https://www.youtube.com/watch?v=CEz1EeDlpbs>
- <https://programarfacil.com/blog/motor-paso-a-paso/> **** OK descripciones y códigos sin y con librería ****
- <https://coeleveld.com/arduino-stepper-uln2003a/> **** Varios ejemplos ****
- <https://www.youtube.com/watch?v=eyqwLiowZiU> Animaciones de funcionamiento del motor
- Características ULN2003A: <https://www.ti.com/lit/ds/symlink/uln2003a.pdf>
- 28BYJ-48 stepper motor and ULN2003 Arduino. Quick tutorial for beginners: <https://www.youtube.com/watch?v=avrdDZD7qEQ>
- Motores unipolares y bipolares: <https://blog.330ohms.com/2016/02/09/motores-a-pasos-unipolares-o-bipolares/>
- <https://www.youtube.com/watch?v=fvBVF7fY0r4> Tut para utilizar stepper sin driver.

20

Relé

Caps 22 Libro de Proyectos de Elegoo

Índice

- Materiales:
 - Relé
 - Motor DC y Batería u otro aparato eléctrico AC
- Circuito
- Actividad
- Referencias

Relé

Es un interruptor que se puede conmutar con una señal eléctrica.

Suelen tener en su interior un **electroimán** que controla un interruptor, aunque también existen relés basados en semiconductores (**relé de estado sólido**)

Permiten **controlar (encender y apagar) tensiones y corrientes muy superiores** a las que permitiría la electrónica de Arduino.

El tiempo de funcionamiento (desde 100 milisegundos hasta 2 segundos) es algo elevado comparado con los tiempos de los circuitos electrónicos, por lo que no se puede utilizar con señales PWM. Los relés de estado sólido son más rápidos y duraderos (no tienen partes móviles)

Muy versátiles: conectándolo al cable de alimentación de cualquier electrodoméstico u otro aparato eléctrico podemos controlarlo de forma sencilla.

Muy útil en domótica e IoT: podemos controlar luces, motores, etc con un solo botón (real o virtual)

Uso en **circuitos de protección**, permitiendo aislar circuitos de alto voltaje.

Se utilizaban mucho en **conmutación telefónica analógica**.

También se puede desarrollar **circuitos lógicos** con relés.

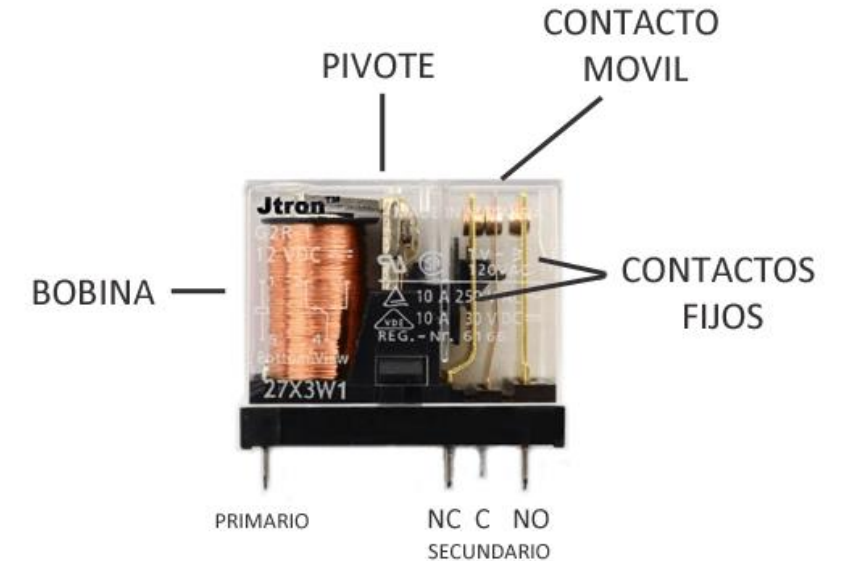


Relé

El relé tiene dos partes

- **Circuito primario:** Se conecta la electrónica de baja tensión (Arduino). Es la parte que recibe la señal de conmutación.
- **Circuito secundario:** Se conecta a la **Carga:** circuito de alta tensión y/o corriente que se desea controlar.
- **C:** Común
- **NO:** Normalmente Open
- **NC:** Normalmente Closed. No existe en todos los modelos

Normalmente el terminal C está en contacto con el NC. Al aplicar corriente en el primario, se mueve el terminal C y se une al NO. Si se apaga la corriente del primario, el terminal C vuelve a su posición de contacto con NC.



Carga: Tensión máxima 250VAC.
Intensidad máxima 2A.

Relé

Al circuito primario hay que conectar GND y un pin digital de Arduino

Al terminal **C** hay que conectar un polo del circuito de carga a controlar.

El otro polo de la carga hay que conectarlo al **NO** o al **NC** dependiendo de si se desea que cuando no haya corriente en el circuito principal la carga este desconectada o activa.

5V Relay Terminals and Pins



Relé

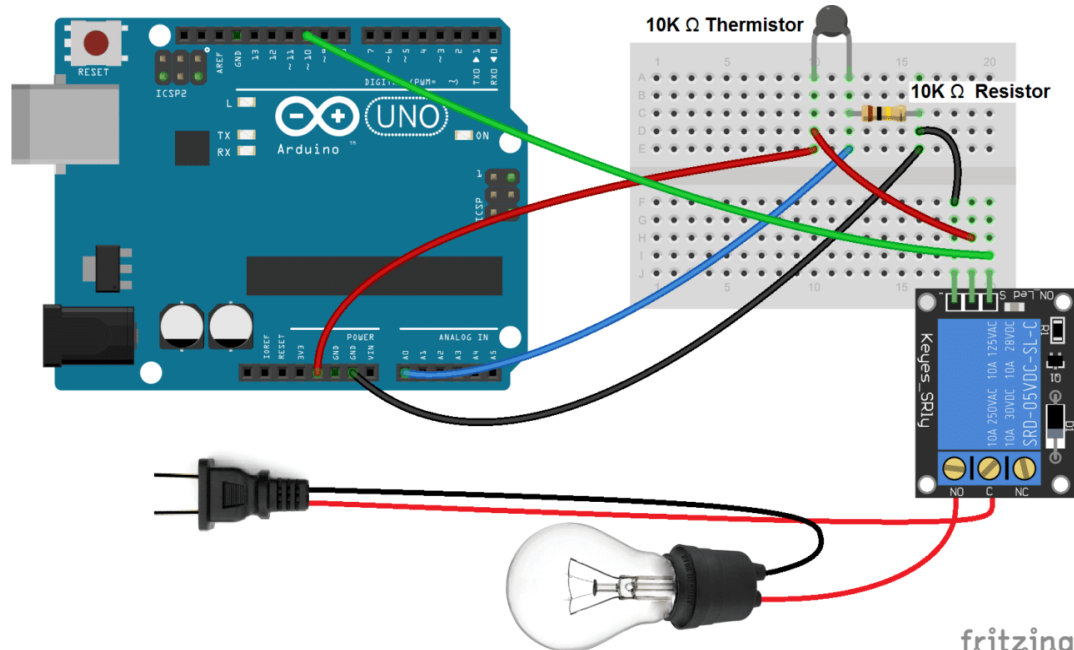
El relé de la figura permite conmutar:

- Corrientes hasta 10A
- Voltajes DC: hasta 30V
- Voltajes AC: hasta 250V

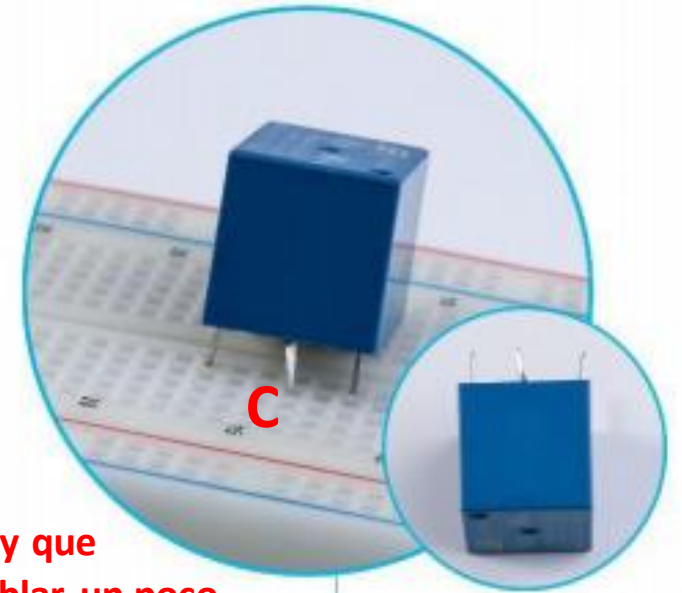


NO SE PONE LA TIERRA EN COMÚN!!!

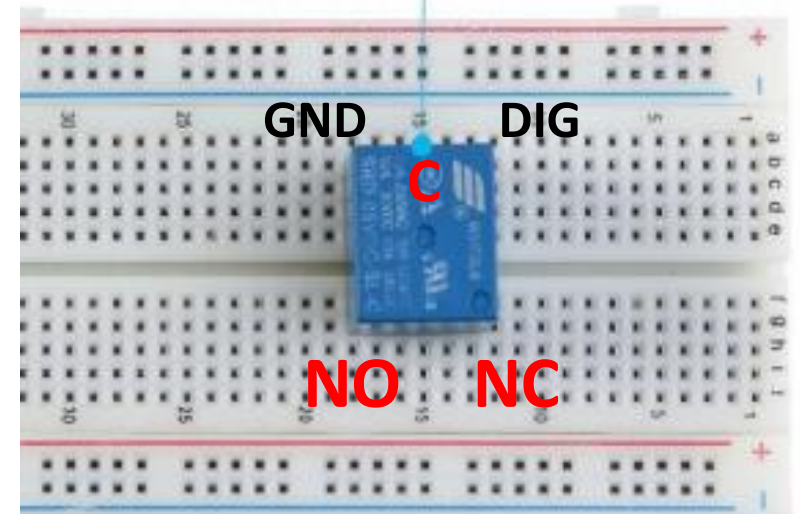
SON CIRCUITOS INDEPENDIENTES



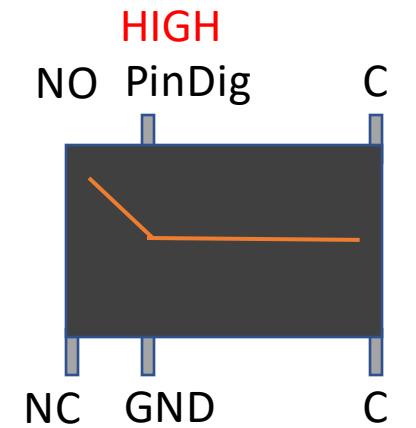
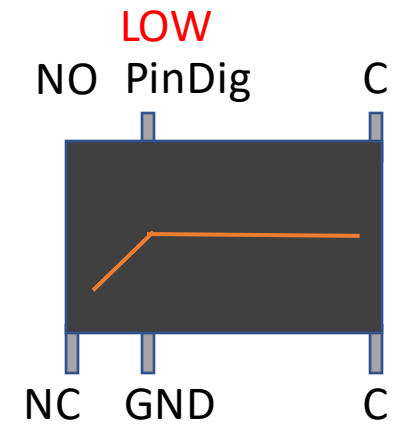
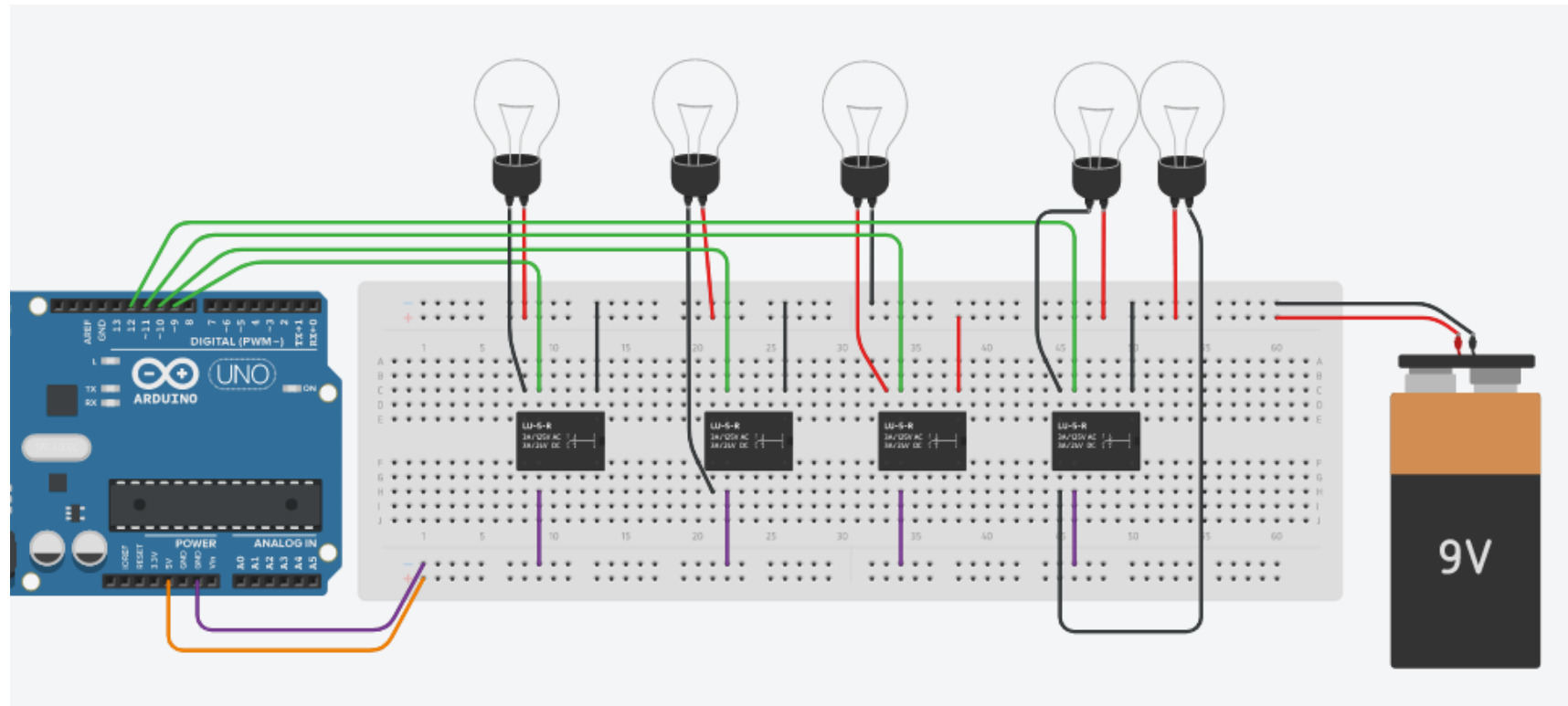
fritzing



Hay que
doblar un poco
la pata central!



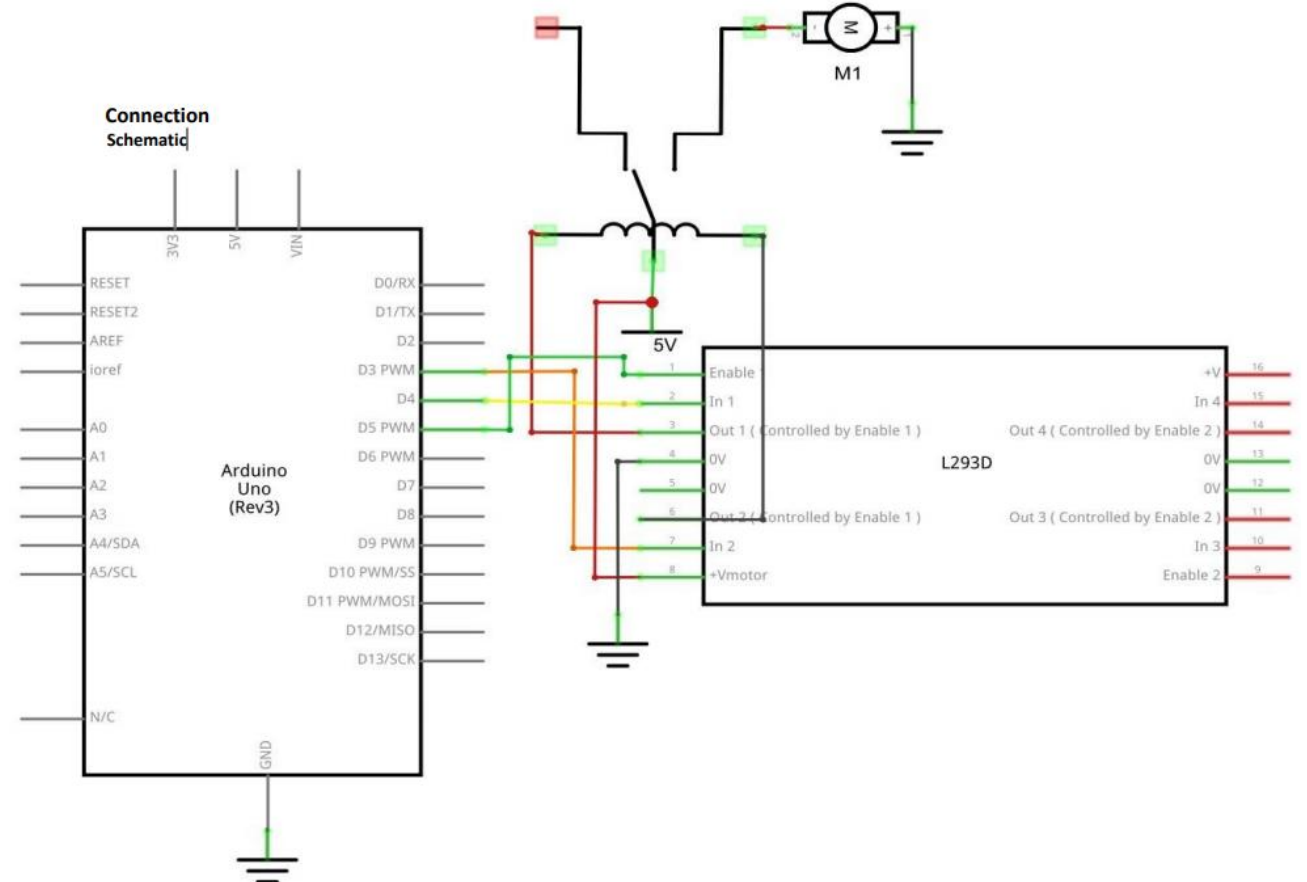
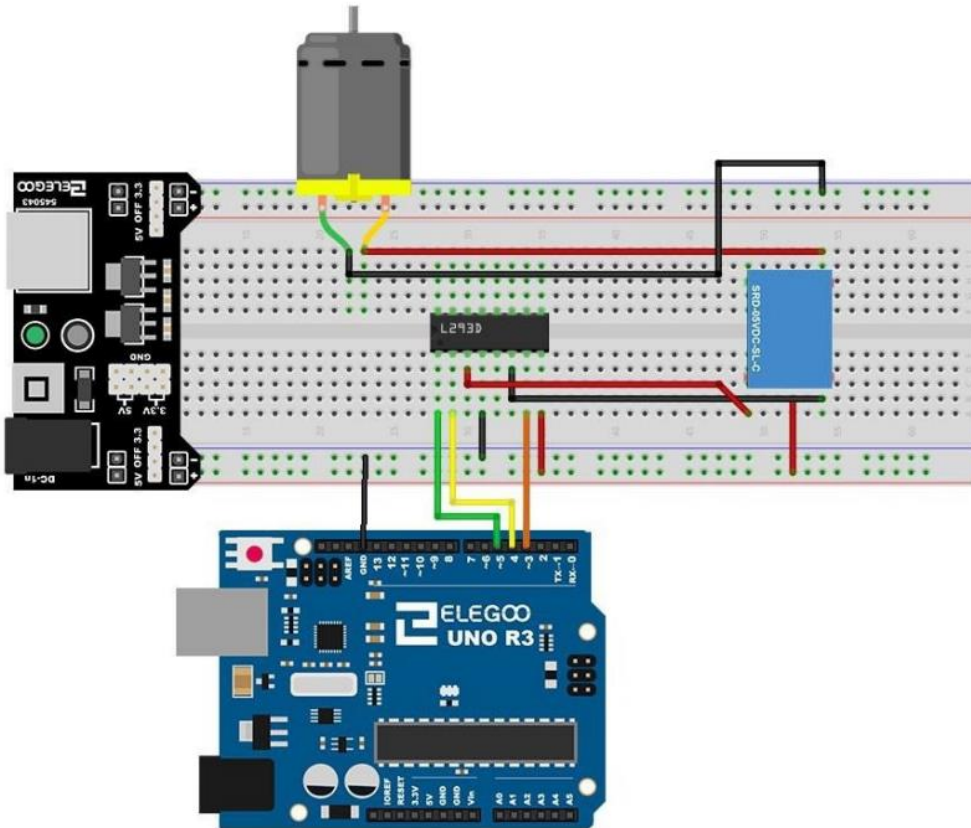
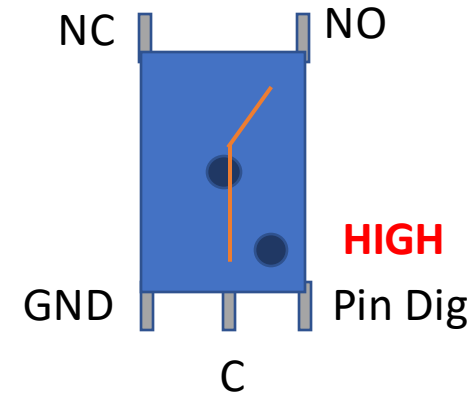
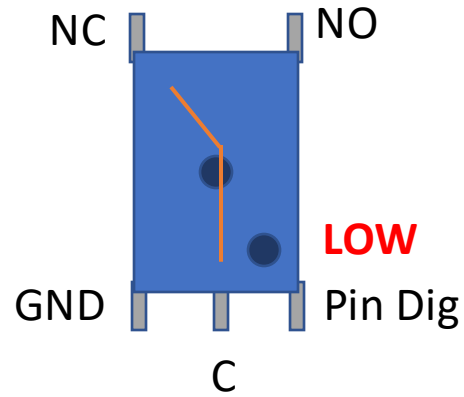
Relé Tinkercad



Circuito ejemplo

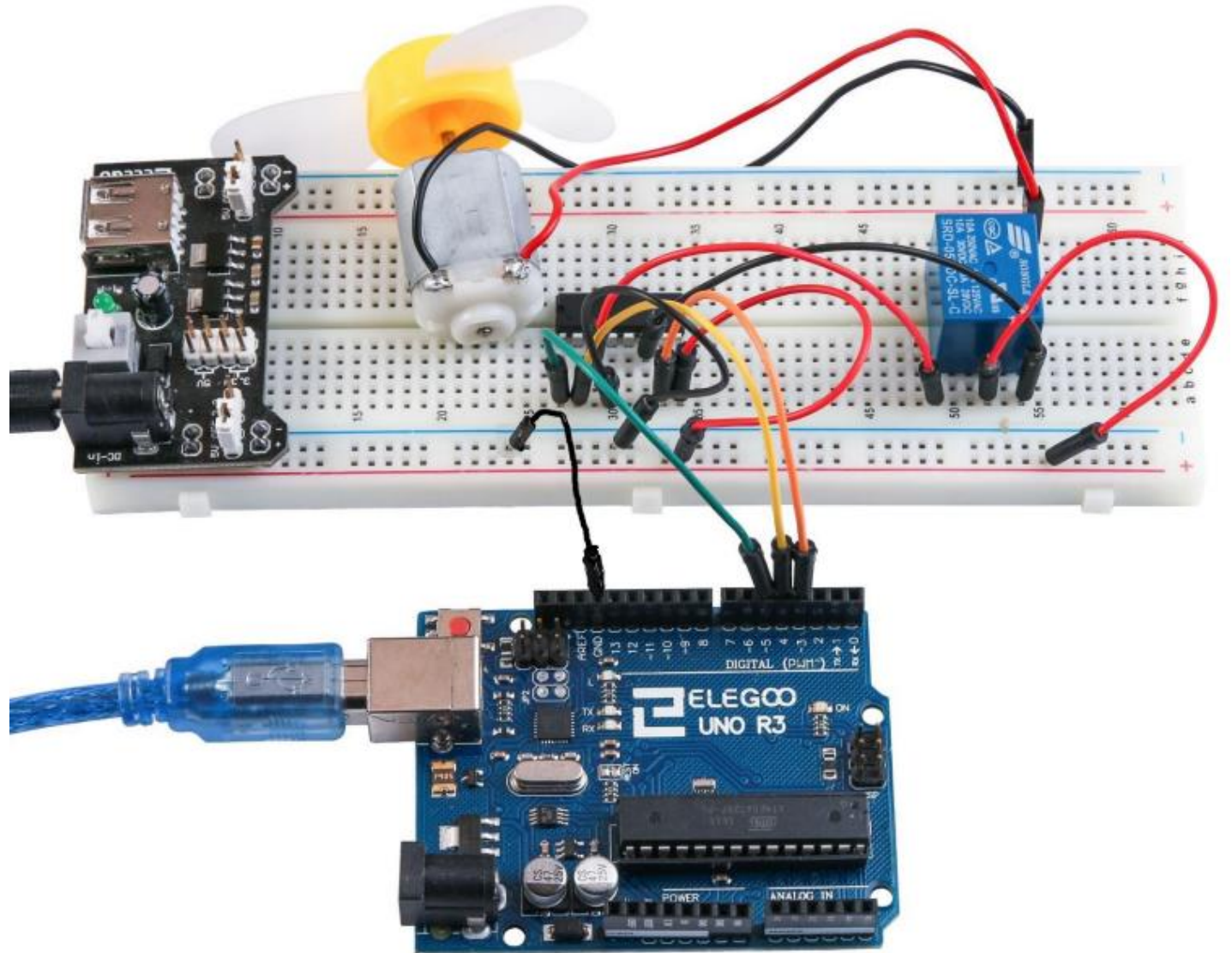
NO SE PONE LA TIERRA EN COMÚN!!!

SON CIRCUITOS INDEPENDIENTES

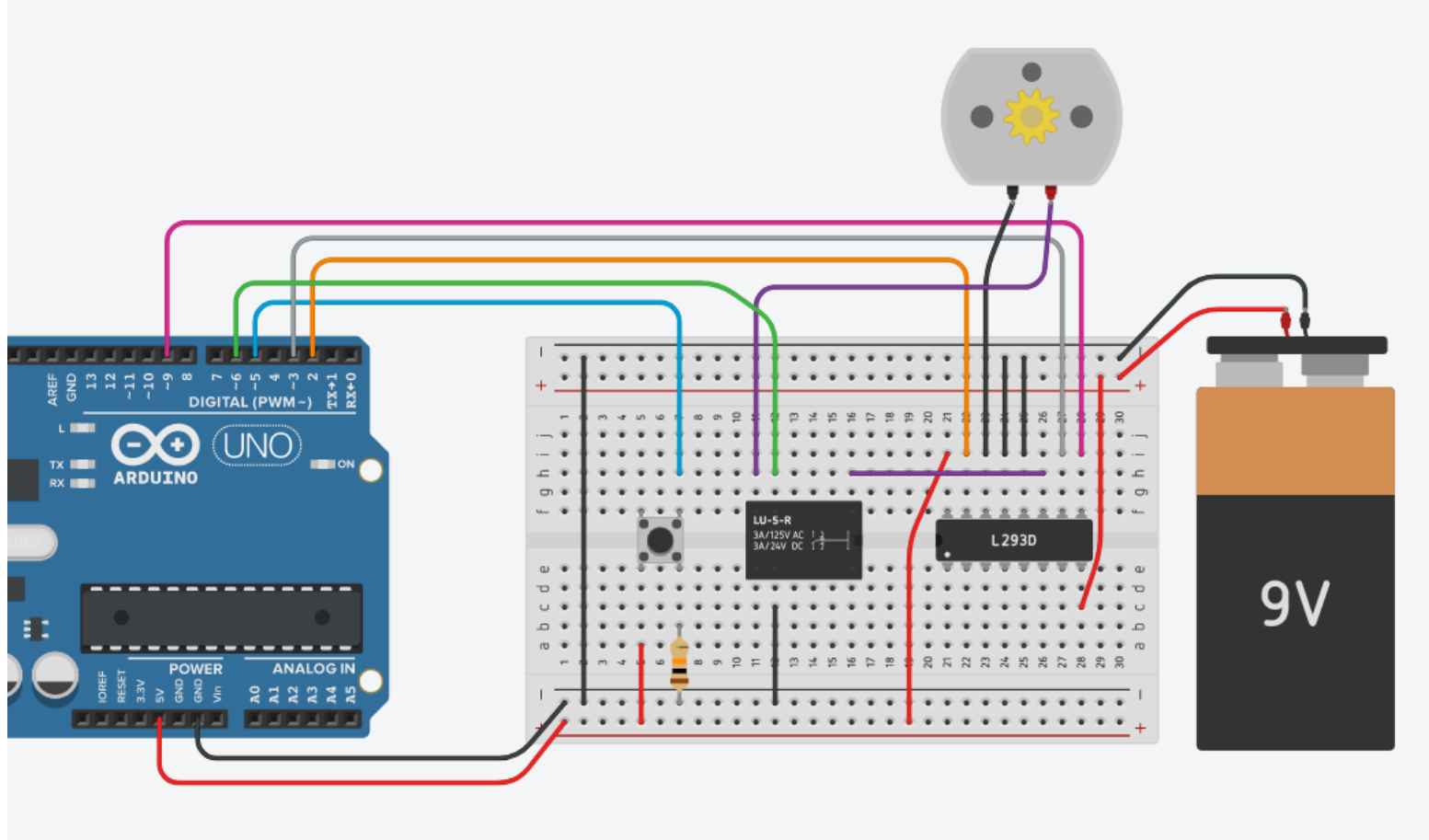


Circuito Ejemplo

En este circuito el relé lo controlarían las salidas del controlador del motor. Dependiendo de la polaridad se activaría o no el relé.



Circuito



Código

- El código es muy sencillo. Basta con controlar el pin digital conectado al relé para conectar o desconectar el circuito controlado por el relé.

```
void setup()
```

```
    pinMode(9, OUTPUT);
```

```
void loop()
```

```
    digitalWrite(9, HIGH);
```

```
    delay(1000);
```

```
    digitalWrite(9, LOW);
```

```
    delay(1000);
```


Actividad

- Montar un circuito que controle un sistema el encendido y apagado de un **motor** (controlado por un L923D), o de un aparato eléctrico de casa (corriente AC), una lámpara por ejemplo.
- Montar un **pulsador** conectado a un pin digital.
- **Programar** que con el botón el motor se encienda o apague mediante la acción del relé.
- **Programar** que con una orden via serial el motor se encienda o apague mediante la acción del relé.
- **XTRA:** ¿Ya lo podéis unir a Processing?

Referencias

- Relé: <https://www.luisllamas.es/arduino-salida-rele/>
- Relés de estado sólido: <https://www.luisllamas.es/arduino-rele-estado-solido-ssr/>
- Relé y Arduino: <https://www.circuitbasics.com/setting-up-a-5v-relay-on-the-arduino/>
- Conmutación con relés: <https://www.oposinet.com/temario-de-tecnologia/temario-1-tecnologia/tema-59-circuitos-de-conmutacin-con-rels/>
- Circuitos lógicos conmutados: <https://unicrom.com/circuitos-logicos-sistemas-digitales/>